MLSMTP

Instructions

Running the program

To run the program, run the `mlsmtpnewinstance` file to initialize config in a given directory. Then, run `mlsmtpd --conf <path>` where `<path>` is the path to `default.json` in the directory.

Input

Input can be provided in three ways:

Submitted emails

To submit emails, connect an email client to the server on the configured port. Then, you can send an email to the server, submitting it via API. Alternatively, you can send an email to an address where this server is specified in the MX record, such that another server provides input via API.

Config files

Input can be provided via config files. These are the various JSON files created when you make a new instance. Input is provided my modifying them according to what is needed such that the program executes properly.

Queue and list commands

Input can be provided via the `mlsmtpqueue` and `mlsmtplist` commands. These commands allow you to remove and modify the contents of the mail queue as specified in the help menu. Additionally, you can use the `mlsmtplist` command to add and remove email lists and addresses to and from those lists according to their help menus.

Output

Output can be provided in two ways:

Sent emails

Output can be given in the form of emails delivered. These emails can originate either from users and servers, or as errors generated by the server. They can be delivered locally, to a local storage option specified, or remotely, using SMTP to deliver emails via API to an external server.

Logs

When you run any of the mlsmtp executables, they will provide constant log output according to the settings. This will contain information on the status, operation, and actions of the server or the program you ran. An example is below:

```
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Created server 1680 on 0.0.0.0:2525 with encryption starttls
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Started server 1680 PID: 34535
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Created server 1700 on 0.0.0.0:5870 with encryption starttls
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Started server 1700 PID: 34536
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Created server 1720 on 0.0.0.0:4650 with encryption implicit
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Started server 1720 PID: 34537
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Created worker 0 (0:1)
[04/16/2025 09:31 PDT] [INFO] [Daemon]: Started worker 0 PID: 34538
[04/16/2025 09:31 PDT] [INFO] [Server 1700]: Successfully connected to database
[04/16/2025 09:31 PDT] [INFO] [Server 1720]: Successfully connected to database
[04/16/2025 09:31 PDT] [INFO] [Server 1700]: Listening on 0.0.0.0:5870, encryption: starttls
[04/16/2025 09:31 PDT] [INFO] [Server 1680]: Successfully connected to database
[04/16/2025 09:31 PDT] [INFO] [Server 1720]: Listening on 0.0.0.0:4650, encryption: implicit
[04/16/2025 09:31 PDT] [INFO] [Worker 0]: Connected to database
[04/16/2025 09:31 PDT] [INFO] [Server 1680]: Listening on 0.0.0.0:2525, encryption: starttls
[04/16/2025 09:31 PDT] [INFO] [Worker 0]: Handling queued jobs (0:1)
^C[04/16/2025 09:31 PDT] [INFO] [Daemon]: Shutting down...
[04/16/2025 09:31 PDT] [INFO] [Server 1680]: Stopped server
[04/16/2025 09:31 PDT] [INFO] [Server 1680]: Killed server
[04/16/2025 09:31 PDT] [INFO] [Server 1700]: Stopped server
[04/16/2025 09:31 PDT] [INFO] [Server 1700]: Killed server
[04/16/2025 09:31 PDT] [INFO] [Server 1720]: Stopped server
[04/16/2025 09:31 PDT] [INFO] [Server 1720]: Killed server
[04/16/2025 09:31 PDT] [INFO] [Worker 0]: Stopped worker
[04/16/2025 09:31 PDT] [INFO] [Worker 0]: Killed worker
```

This log output can be given to the terminal, or to a set of specified files. Additionally, when using STDOUT in terminal, logs can be color coded for easier recognition of problems.

Code

All files are provided below:

lib/mlsmtp/message_authorization/dkim/dkim_signature_header.rb

```ruby
module SMTPServer
  # Defines the SMTPServer module which contains message authorization functionalities
  module MessageAuthorization
    # Defines the DKIMSignatureHeader class responsible for creating DKIM signature headers
    class DKIMSignatureHeader
      # Loads DKIM maps from a JSON file specified in the configuration at class load time
      @@dkim_maps = JSON.parse(
        File.read(
          Config.active["dkim_maps"]
        )
      )

      # Initializes a new DKIMSignatureHeader object with the given context
      # Input: context - object containing email data and logger information
      def initialize(context)
        # Extracts the domain part of the sender email address
        @email_domain = Transport::Destination.new(context.mailfrom, get_servers: false).addr_domain
        # Retrieves DKIM configuration map for the email domain
        map = @@dkim_maps[@email_domain]

        if map
          # Logs attempt to sign message with DKIM
          Logger.log "Attempting to sign with DKIM", origin: context.logger_origin, verbosity: 5

          # Reads and encodes the email message from the context data
          @message_encoded = Mail.read_from_string(context.data).encoded
          # Retrieves the private key associated with the keypair specified in the map
          @key = SSL::Certificate[map["keypair"]].key
          # Sets the domain to be used in the DKIM signature
          @domain = map["domain"]
```

```
          # Ensures the selector is an array for random selection if multiple are provided
          map["selector"] = [ map["selector"] ] if map["selector"].is_a?(String)
          # Randomly selects a selector if multiple are available
          @selector = map["selector"].length == 1 ? map["selector"][0] : map["selector"][rand(0...map["selector"].length)]

          begin
            # Creates a new DKIM signed mail object to generate the DKIM header
            @signature_header = Dkim::SignedMail.new(
              @message_encoded,
              domain: @domain,
              selector: @selector,
              private_key: @key
            )
              # Retrieves the DKIM header value from the signed mail object
              .dkim_header
              .value

            # Logs successful signing of the message
            Logger.log "Message signed successfully", origin: context.logger_origin, verbosity: 5
          rescue => e
            # Handles exceptions during signing process and logs warnings
            Logger.log "Error signing message: #{e}", origin: context.logger_origin, verbosity: 5, type: :warn
            @signature_header = nil
          end
        else
          # Logs warning if no DKIM settings are found for the email domain
          Logger.log "Domain `#{@email_domain}` has no DKIM settings", origin: context.logger_origin, verbosity: 5, type: :warn
        end
      end

      # Converts the DKIM signature header to string format
      def to_s
        @signature_header
      end
    end
  end
end
```

lib/mlsmtp/message_authorization/spf/authorize_spf.rb

```
# This module defines a namespace for SMTP message authorization related to SPF authentication.
module SMTPServer
  module MessageAuthorization
    module SPFAuthenticator
      # This method performs SPF authorization for a given email context and configuration.
      # Inputs:
      # - context: an object containing email transaction details such as mailfrom, ip_addr, heloname, and logger_origin.
      # - config: a hash containing configuration parameters including versions, on_spf_fail, and ok_results.
      # Output:
      # - returns true if the SPF result is considered successful based on configuration, false otherwise.
      def self.authorize_spf(context, config)
        # Log the start of SPF authorization attempt with specified verbosity.
        Logger.log "Attempting SPF authorization", origin: context.logger_origin, verbosity: 5

        # Initialize a new SPF server instance to process SPF requests.
        spf_server = SPF::Server.new

        begin
          # Create an SPF request object with version info, scope, identity, IP address, and HELO identity.
          request = SPF::Request.new(
            versions: config["versions"], # SPF versions to use.
            scope: "mfrom",              # Scope of the SPF check, here mail from.
            identity: Transport::Destination.new(context.mailfrom, get_servers: false).address, # Sender email address.
            ip_address: context.ip_addr, # IP address of the connecting client.
            helo_identity: context.heloname # HELO/EHLO identity.
          )

          # Process the SPF request using the SPF server.
          result = spf_server.process(request)

          # Convert the result code to string for comparison.
          result_code = result.code.to_s
        rescue
          # In case of any exception during SPF processing, assign the failure code from config.
          result_code = config["on_spf_fail"]
        end

        # Log the SPF result code with specified verbosity.
        Logger.log "SPF result: #{result_code}", origin: context.logger_origin, verbosity: 5

        # Store the SPF result code in the context's additional authorization data.
        context.additional_authorization_data[:spf_result] = result_code

        # Determine if the SPF result is acceptable based on configured OK results.
        return config["ok_results"].include?(result_code)
      end
    end
  end
end
```

lib/mlsmtp/message_authorization/spf/header.rb

```ruby
# Define the SMTPServer module
module SMTPServer
  # Define the nested MessageAuthorization module within SMTPServer
  module MessageAuthorization
    # Define the SPFHeader class responsible for generating SPF header strings
    class SPFHeader
      # Initialize the SPFHeader object with context data
      # Inputs:
      #   context - an object containing authorization and connection details
      def initialize(context)
        # Store whether the message is exempt from authorization checks
        @auth_exempt = context.authorization_exempt
        # Store the SPF result obtained from additional authorization data
        @spf_result = context.additional_authorization_data[:spf_result]
        # Store the client's IP address
        @client_ip = context.ip_addr
        # Store the hostname used in the HELO/EHLO command
        @heloname = context.heloname
        # Store the envelope sender address
        @mailfrom = context.mailfrom
      end

      # Generate the SPF header string if not exempt
      # Returns:
      #   nil if the message is exempt, otherwise the formatted SPF header string
      def to_s
        # Check if the message is exempt from authorization
        if @auth_exempt
          # Return nil if exempt, indicating no SPF header should be added
          nil
        else
          # Construct and return the SPF header string with relevant details
          "Received-SPF: #{@spf_result} (mailfrom) identity=mailfrom; client-ip=#{@client_ip}; helo=#{@heloname}; envelope-from=#{@m
        end
      end
    end
  end
end
```

lib/mlsmtp/queue/queue_handler.rb

```ruby
# Define the SMTPServer module to encapsulate related classes and modules
module SMTPServer
  # Define the Queue module to handle message queuing functionalities
  module Queue
    # Define the QueueHandler class responsible for processing queued messages
    class QueueHandler
      # Class variables for delivery timeout and retry interval, loaded from configuration
      @@delivery_timeout = Config.active["transport"]["delivery_timeout"]
      @@retry_interval = Config.active["transport"]["retry_interval"]

      # Initialize the QueueHandler with module and queue identifiers
      # Inputs:
      # - mod, eq: Specified so it can find messages where Message ID % MOD == EQ
      def initialize(mod:, eq:)
        @mod = mod
        @eq = eq
        # Set the origin string for logging purposes
        @origin = "Worker #{eq}"
      end

      # Main method to run the queue processing loop
      def run
```

```
        # Infinite loop to continuously process queued messages
        while true
          # Retrieve all queued messages for the specified module and queue
          queued_messages = QueuedMessage.find_by_mod(mod: @mod, eq: @eq)
          # Iterate over each message in the queue
          queued_messages.each do |message|
            # Unpack message attributes
            mid, uid, is_error_response, created_at, mail_from, rcpt_to, file_path, retries, try_at = message
            # Skip message if it's scheduled for a future attempt
            next if try_at > Time.now.to_i

            # Create an origin string for logging that includes the unique identifier
            origin = "Worker #{@eq}: #{uid}"

            # Log attempt to deliver the message
            Logger.log "Attempting to deliver queued message to `#{rcpt_to}`", origin: origin, verbosity: 2

            # Create a destination object for the recipient address
            destination = Transport::Destination.new(rcpt_to)

            # Read the message data from the file system
            message_data = File.read(file_path)

            # Check if the destination is local
            if destination.local
              # Log local delivery attempt
              Logger.log "Destination is local, using local delivery agent", origin: origin, verbosity: 3

              # Instantiate a local delivery agent with user and message data
              agent = Transport::LocalDeliveryAgent.new(
                user: destination.destination_user,
                message: message_data
              )
            else
              # Log remote delivery attempt
              Logger.log "Destination is remote, using remote delivery agent", origin: origin, verbosity: 3

              # Instantiate a remote delivery agent with message, destination, sender, and origin info
              agent = Transport::RemoteDeliveryAgent.new(
                message: message_data,
                destination: destination,
                sender: mail_from,
                origin: origin
              )
            end

            begin
              # Attempt delivery within the specified timeout
              Timeout.timeout(@@delivery_timeout) do
                agent.attempt_delivery
                # Log success message upon successful delivery
                Logger.log "Message delivered successfully to #{"mailbox " if destination.local}`#{destination.destination_user}`",
              end
            rescue => e
              # Handle delivery failure exceptions
              if retries <= 0
                # Generate and queue an error email if no retries remain
                generator = Email::ErrorEmailGenerator.new(
                  e,
                  origin: origin,
                  mail_from: mail_from,
                  rcpt_to: rcpt_to,
                  is_error_response: is_error_response
                )
                generator.queue_email
              else
```

```
                # Log the error and remaining retries
                Logger.log "Error delivering email: `#{e}`, trying #{retries} more time#{?s unless retries == 1}", origin: origin, v

                # Create a new queued message for retry with decremented retries and scheduled try time
                retry_message = QueuedMessage.new(
                  mail_from: mail_from,
                  rcpt_to: rcpt_to,
                  message: message_data,
                  retries: retries - 1,
                  try_at: Time.now.to_i + @@retry_interval
                )

                # Retrieve the queue identifier for the retry message
                quid = retry_message.queue[1]
                # Log the queued retry message
                Logger.log "Queued retry message as #{quid}", origin: origin, verbosity: 3
              end
            end

            # Remove the processed message from the queue based on its unique identifier
            QueuedMessage.unqueue_uid(uid)
            # Log the removal of the message from the queue
            Logger.log "Removed message #{uid} from queue", origin: origin, verbosity: 3
          end
        end
      end
    end

    # Provide read access to module and eq attributes
    attr_reader :mod, :eq
  end
end
```

lib/mlsmtp/queue/queued_message.rb

```
# Define the SMTPServer module to encapsulate server related classes and modules
module SMTPServer
  # Define the Queue module to handle message queuing functionalities
  module Queue
    # Define the QueuedMessage class to represent and manage queued email messages
    class QueuedMessage
      # Retrieve the directory path for queued mail from configuration
      queue_dir = Config.active["queue"]["queued_mail_dir"]
      # Create the directory if it does not exist to store queued emails
      Dir.mkdir(queue_dir) unless File.exist?(queue_dir)
      # Set the default number of retries for message delivery from configuration
      @@default_retries = Config.active["transport"]["max_retries"]

      # Initialize a new QueuedMessage instance with required attributes
      # Inputs:
      # - mail_from: sender email address
      # - rcpt_to: recipient email address
      # - message: email message content
      # - retries: number of delivery retries (optional, defaults to class variable)
      # - try_at: timestamp to attempt delivery (optional)
      # - error_response: flag indicating if this is an error response (optional)
      def initialize(mail_from:, rcpt_to:, message:, retries: @@default_retries, try_at: nil, error_response: false)
        @mail_from = mail_from
        @rcpt_to = rcpt_to
        @message = message
        @message_id = nil
        @queued = false
        @error_response = error_response
        @retries = retries
```

```
    @retries = retries
    # Set the attempt time to provided try_at or current time if nil
    @try_at = try_at ? try_at : Time.now.to_i
  end

  # Queue the message for delivery by saving to file and database
  # Returns array of file path and message UID if successful, false otherwise
  def queue
    # Return false if database connection is not active
    return false unless Database.active

    # Proceed only if message is not already queued
    unless @queued
      # Generate file path and message UID
      @file_path, @message_uid = get_path_and_id

      # Write message content to the file system at the generated path
      File.write(@file_path, @message)

      # Insert message metadata into the database
      Database.active.exec_sql *build_sql(
        mail_from: @mail_from,
        message_uid: @message_uid,
        rcpt_to: @rcpt_to,
        message: @message,
        file_path: @file_path,
        is_error_response: @error_response ? 1 : 0,
        retries: @retries,
        try_at: @try_at
      )

      # Mark message as queued to prevent duplicate queuing
      @queued = true
    end

    # Return the file path and message UID for reference
    return [ @file_path, @message_uid ]
  end

  # Class method to find messages by message ID modulo a number
  # Inputs:
  # - mod: divisor for modulo operation (default 1)
  # - eq: expected remainder (default 0)
  # Returns database records matching the condition or nil if database inactive
  def self.find_by_mod(mod: 1, eq: 0)
    # Return nil if database is inactive
    return nil unless Database.active

    # Execute SQL to select messages where message_id modulo mod equals eq
    Database.active.exec_sql(
      "SELECT * FROM queued_messages WHERE message_id % ? = ?",
      [
        mod,
        eq
      ]
    )
  end

  # Class method to find a message by its unique message ID
  # Input:
  # - mid: message ID to search for
  # Returns matching database record or nil if database inactive
  def self.find_by_mid(mid)
    # Return nil if database is inactive
    return nil unless Database.active
```

```
    # Execute SQL to select message with specific message ID
    Database.active.exec_sql(
      "SELECT * FROM queued_messages WHERE message_id = ?",
      mid
    )
  end

  # Class method to find a message by its unique message UID
  # Input:
  # - uid: message UID to search for
  # Returns matching database record or nil if database inactive
  def self.find_by_uid(uid)
    # Return nil if database is inactive
    return nil unless Database.active

    # Execute SQL to select message with specific message UID
    Database.active.exec_sql(
      "SELECT * FROM queued_messages WHERE message_uid = ?",
      uid
    )
  end

  # Class method to remove a message from the queue using its UID
  # Input:
  # - uid: message UID to unqueue
  # Returns nil if database inactive or no messages found
  def self.unqueue_uid(uid)
    # Return nil if database is inactive
    return nil unless Database.active
    # Find messages with the given UID
    messages = find_by_uid(uid)
    # Return if no messages found with that UID
    return if messages.empty?

    # Remove associated file if configuration allows removal on unqueue
    if Config.active["queue"]["remove_on_unqueue"]
      # Get the file path from the first message record
      path = messages[0][6]
      # Delete the file if it exists
      File.delete(path) if File.exist?(path)
    end

    # Delete the message record from the database
    Database.active.exec_sql(
      "DELETE FROM queued_messages WHERE message_uid = ?",
      uid
    )
  end

  # Attribute readers for instance variables
  attr_reader :mail_from, :rcpt_to, :message, :queued, :message_id, :file_path, :message_uid

  private

  # Helper method to build SQL insert statement and parameters
  # Inputs:
  # - message_uid: unique message identifier
  # - is_error_response: flag if message is an error response
  # - mail_from: sender email address
  # - rcpt_to: recipient email address
  # - message: email message content
  # - file_path: path to stored message file
  # - retries: number of retries remaining
  # - try_at: timestamp to attempt delivery
  # Returns array containing SQL statement and parameters
```

```
      def build_sql(message_uid:, is_error_response: 0, mail_from:, rcpt_to:, message:, file_path:, retries:, try_at:)
        [
          "INSERT INTO queued_messages (message_uid, is_error_response, created_at, mail_from, rcpt_to, file_path, retries, try_at)
          [
            message_uid,
            is_error_response,
            Time.now.to_i,
            mail_from,
            rcpt_to,
            file_path,
            retries,
            try_at,
          ]
        ]
      end

      # Helper method to generate a unique file path and message UID
      # Loops until a unique file path (not existing) is found
      # Returns array with file path and message UID
      def get_path_and_id
        loop do
          # Generate a random 13-digit number as message UID
          message_uid = rand(1000000000000..9999999999999)
          # Construct file path using configuration directory and UID
          path = Config.active["queue"]["queued_mail_dir"] + "/#{message_uid}.eml"
          # Return path and UID if file does not already exist
          return [ path, message_uid ] unless File.exist?(path)
        end
      end
    end
  end
end
```

lib/mlsmtp/queue/queue_worker.rb

```
# Module defining the SMTPServer namespace
module SMTPServer
  # Module defining the Queue namespace within SMTPServer
  module Queue
    # Class representing a worker that processes queued email jobs
    class QueueWorker
      # Class variable holding all worker instances
      @@all_workers = []
      # Class variable holding currently running worker instances
      @@running_workers = []

      # Initialize a new QueueWorker with specified module and eq identifiers
      # Inputs:
      # - mod: the module name associated with the worker
      # - eq: the queue identifier
      def initialize(mod:, eq:)
        @mod = mod  # Store module name
        @eq = eq    # Store queue identifier

        @running = false  # Flag indicating if worker is running
        @dead = false     # Flag indicating if worker is dead/killed
        @pid = nil        # Process ID of the forked worker process

        @origin = "Worker #{eq}"  # String identifying the worker origin
        @@all_workers << self     # Add this instance to all workers list
      end

      # Start the worker process if not already running or dead
      # Returns false if worker is dead, true if already running, or the process ID if started
```

```
    def start
      return false if @dead  # Do not start if worker is dead
      return true if @running  # Do nothing if already running

      @running = true  # Mark worker as running
      @@running_workers << self  # Add to running workers list

      # Fork a new process to handle queue jobs
      @pid = fork do
        Signal.trap("INT", "IGNORE")  # Ignore interrupt signals in child process

        begin
          Database.connect  # Connect to the database
          Logger.log "Connected to database", origin: @origin, verbosity: 1  # Log success
        rescue => e
          Logger.log "Failed to connect to database", origin: @origin, verbosity: 1, type: :error  # Log failure
          raise e  # Re-raise exception to terminate process
        end

        # Log that worker is handling queued jobs
        Logger.log "Handling queued jobs (#{@eq}:#{@mod})", origin: @origin, verbosity: 1
        # Instantiate a QueueHandler with module and eq
        handler = QueueHandler.new(mod: @mod, eq: @eq)
        handler.run  # Run the handler to process jobs
      end

      @pid  # Return process ID of the forked process
    end

    # Restart the worker process after stopping it
    def restart
      stop  # Stop current worker process
      sleep(Config.active["queue"]["workers"]["restart_delay"])  # Wait for configured delay
      start  # Start a new worker process
    end

    # Stop the worker process gracefully
    # Inputs:
    # - killcode: the signal to send for termination, default is TERM
    def stop(killcode: "TERM")
      Process.kill(killcode, @pid)  # Send termination signal to process
      @running = false  # Mark worker as not running
      @@running_workers.delete(self)  # Remove from running workers list

      Logger.log "Stopped worker", origin: @origin, verbosity: 1  # Log stopping
    end

    # Kill the worker process and mark as dead
    def kill
      stop  # Stop the process
      @dead = true  # Mark worker as dead
      @@all_workers.delete(self)  # Remove from all workers list

      Logger.log "Killed worker", origin: @origin, verbosity: 1  # Log killing
    end

    # Class method to retrieve all worker instances
    def self.all_workers
      @@all_workers
    end

    # Class method to retrieve only currently running worker instances
    def self.running_workers
      @@running_workers
    end
```

```
      # Attribute readers for worker properties
      attr_reader :mod, :eq, :running, :dead, :pid
    end
  end
end
```

lib/mlsmtp/servers/mailserver.rb

```
# This module defines the SMTPServer namespace and nested Servers module containing the MailServer class
module SMTPServer
  module Servers
    # The MailServer class models an SMTP server with start, stop, restart, and kill capabilities
    class MailServer
      # Class variables to keep track of all server instances and currently running servers
      @@all_servers = []
      @@running_servers = []

      # Initialize a new MailServer instance with host, port, encryption type, and certificate
      # Inputs:
      #   host - the hostname or IP address to listen on
      #   port - the port number to listen on
      #   encryption - symbol indicating encryption type (:implicit or :starttls)
      #   certificate - SSL certificate object for encryption
      def initialize(host:, port:, encryption:, certificate:)
        @host = host
        @port = port
        @running = false # Indicates if server is running
        @dead = false    # Indicates if server has been killed
        @pid = nil       # Process ID of the server process
        @encryption = encryption
        @certificate = certificate

        # Generate a unique origin string for logging purposes
        obj_id = self.object_id
        @origin = "Server #{obj_id}"

        # Add this server instance to the list of all servers
        @@all_servers << self
      end

      # Starts the server if not already running or dead
      # Returns false if server is dead, true if already running, or process ID if started successfully
      def start
        return false if @dead
        return true if @running

        @running = true
        # Add this server to the list of running servers
        @@running_servers << self

        # Fork a new process to run the server
        @pid = fork do
          # Ignore interrupt signals in the child process
          Signal.trap("INT", "IGNORE")

          begin
            # Connect to the database, raising an error if it fails
            Database.connect
            # Log successful database connection
            Logger.log "Successfully connected to database", origin: @origin, verbosity: 1
          rescue => e
            # Log database connection error
            Logger.log "Error connecting to database", type: :error, origin: @origin
            raise e
```

```
    end

    # Create a TCP server socket listening on specified host and port
    tcp_server = TCPServer.new(@host, @port)

    # Wrap TCP server with SSL if encryption is implicit
    if @encryption == :implicit
      context = @certificate.context
      server = OpenSSL::SSL::SSLServer.new(tcp_server, context)
    else
      server = tcp_server
    end

    # Log listening status with host, port, and encryption info
    Logger.log "Listening on #{@host}:#{@port}, encryption: #{@encryption}", origin: @origin, verbosity: 1

    # Main server loop to accept and handle client connections
    while @running
      begin
        # Accept incoming client connection and handle in a new thread
        Thread.start(server.accept) do |client|
          # Retrieve client's IP address
          client_ip = client.peeraddr[3]

          # Log new client connection
          Logger.log "New connection from #{client_ip}", origin: @origin, verbosity: 2

          # Initialize SMTP client context with the client socket
          context = SMTP::SMTPClientContext.new(client)
          # Enable STARTTLS support if specified
          context.starttls_support = true if @encryption == :starttls
          # Assign certificate context if available
          context.starttls_certificate = @certificate&.context

          # Log creation of SMTP client context
          Logger.log "Creating SMTP client context for #{client_ip}", origin: @origin, verbosity: 3

          # Log handling of client connection
          Logger.log "Handling client #{client_ip}", origin: @origin, verbosity: 3
          # Instantiate a generic client handler and process the client
          handler = Handlers::GenericClientHandler.new(context)
          handler.handle_client
        end
      rescue OpenSSL::SSL::SSLError
        # Log SSL errors encountered during client handling
        Logger.log "SSL error for client", origin: @origin, verbosity: 2, type: :warn
      end
    end
  end

  # Return the process ID of the forked server process
  return @pid
end

# Restarts the server by stopping it, waiting for a delay, then starting again
def restart
  stop
  sleep(Config.active["socket"]["restart_delay"])
  start
end

# Stops the server process gracefully using specified kill code
# Defaults to TERM signal
def stop(killcode: "TERM")
  Process.kill(killcode, @pid) # Send kill signal to server process
```

```
        @running = false # Mark server as not running
        @@running_servers.delete(self) # Remove from running servers list

        # Log server stop event
        Logger.log "Stopped server", origin: @origin, verbosity: 1
      end

      # Kills the server process and marks it as dead
      def kill
        stop # Call stop to terminate process
        @dead = true # Mark server as dead
        @@all_servers.delete(self) # Remove from all servers list

        # Log server kill event
        Logger.log "Killed server", origin: @origin, verbosity: 1
      end

      # Class method to retrieve all server instances
      def self.all_servers
        @@all_servers
      end

      # Class method to retrieve all currently running server instances
      def self.running_servers
        @@running_servers
      end

      # Read-only accessors for server attributes
      attr_reader :host, :port, :running, :dead, :pid
    end
  end
end
```

lib/mlsmtp/authentication/pam.rb

```
# Define the SMTPServer module which encapsulates server-related functionality
module SMTPServer
  # Define the Authentication module for handling authentication mechanisms
  module Authentication
    # Define the PAMAdapter class to adapt PAM authentication within SMTPServer
    class PAMAdapter
      # Initialize the adapter with configuration options
      # conf: a hash containing configuration parameters
      def initialize(conf)
        # Store the override_service parameter from configuration
        @override_service = conf["override_service"]
      end

      # Authenticate a user with a password
      # user: the username to authenticate
      # password: the password for the user
      # Returns true if authentication succeeds, false otherwise
      def authenticate(user, password)
        # If override_service is specified, pass it to Rpam.auth
        if @override_service
          return Rpam.auth(user, password, service: @override_service)
        else
          # Otherwise, call Rpam.auth without specifying a service
          return Rpam.auth(user, password)
        end
      end
    end
  end
end
```

lib/mlsmtp/authentication/none.rb

```
module SMTPServer  # Defines the main module for the SMTP server components
  module Authentication  # Defines a nested module for handling authentication strategies
    class NoneAdapter  # Defines a class for an authentication adapter that does not authenticate
      # Initializes the NoneAdapter with configuration options, but does not use them
      def initialize(conf)
      end

      # Implements the authenticate method which always returns false
      # Inputs: user - the username attempting to authenticate
      #         password - the password provided by the user
      # Output: always returns false indicating authentication failure
      def authenticate(user, password)
        false
      end
    end
  end
end
```

lib/mlsmtp/authentication/plain_handler.rb

```
module SMTPServer
  module Authentication
    # This method handles the PLAIN authentication mechanism for SMTP.
    # Inputs:
    # - context: an object representing the current SMTP session context.
    # - encoded_credentials: an optional array containing the Base64 encoded credentials.
    # It performs authentication and sends appropriate SMTP responses based on success or failure.
    def self.method_plain_handler(context, encoded_credentials)
      Logger.log "Trying auth PLAIN", origin: context.logger_origin, verbosity: 5
      # Log an attempt to authenticate using PLAIN mechanism with verbosity level 5.

      unless encoded_credentials
        # If credentials are not provided, send a response requesting credentials.
        username_request = SMTP::Response.new(
          status: :positive_intermediate, # SMTP status indicating intermediate positive response.
          category: :authentication,       # Response category for authentication.
          detail: 4                        # Detail code indicating need for credentials.
        )
        context.send_response(username_request)
        # Send the response requesting username and password.

        encoded_credentials = context.read[0]
        # Read the next line from client which should contain the credentials.
      else
        encoded_credentials = encoded_credentials[0]
        # If credentials are provided, extract the first element.
      end

      # Decode the Base64 credentials and split into authorization parts.
      _, username, password = Base64.decode64(encoded_credentials).split("\0")
      # The first element is ignored, second is username, third is password.

      Logger.log "Client credentials received: #{username}:***", origin: context.logger_origin, verbosity: 5
      # Log receipt of credentials with username, masking the password.

      if Active.authenticate(username, password)
        # Authenticate credentials using Active module.
        response = SMTP::Response.new(
          status: :positive_completed, # SMTP status indicating success.
          category: :authentication,
          detail: 5,
          message: "2.7.0 Authentication Successful"
        )
```

```
        )
        # Create a success response message.

        context.authenticated_as = username
        # Save the authenticated username in the context.

        Logger.log "Authenticated successfully", origin: context.logger_origin, verbosity: 5
        # Log successful authentication.
      else
        # If authentication fails, prepare a failure response.
        response = SMTP::Response.new(
          status: :negative_permanent, # SMTP status for permanent failure.
          category: :authentication,
          detail: 5,
          message: "5.7.8 Authentication Failed"
        )
        # Create a failure response message.

        Logger.log "Authentication failed", origin: context.logger_origin, verbosity: 5, type: :warn
        # Log the failure with warning level.
      end

      # Send the authentication response back to the client.
      context.send_response(response)
    end
  end
end
```

lib/mlsmtp/authentication/auth_adapter.rb

```
module SMTPServer
  # Defines a module SMTPServer to encapsulate server-related functionalities
  module Authentication
    # Creates an adapter instance based on configuration settings

    # Retrieves the name of the adapter class from the configuration
    adapter_class_name = Config.active["authentication"]["adapter"]
    # Uses Object.const_get to get the class object from the class name string
    adapter_class = Object.const_get(adapter_class_name)

    # Retrieves the configuration hash for the adapter
    adapter_config = Config.active["authentication"]["config"]
    # Instantiates the adapter class with the configuration hash
    adapter = adapter_class.new(adapter_config)

    # Assigns the adapter instance to a constant Active within the module
    Active = adapter
  end
end
```

lib/mlsmtp/authentication/debug.rb

```ruby
# Define the SMTPServer module to encapsulate server-related classes and modules
module SMTPServer
  # Define the Authentication module for handling authentication mechanisms
  module Authentication
    # Define the DebugAdapter class for debugging authentication processes
    class DebugAdapter
      # Initialize the DebugAdapter with a configuration hash containing credentials
      # conf - a hash with user credentials where keys are usernames and values are passwords
      def initialize(conf)
        # Store the credentials hash in an instance variable for later verification
        @good_credentials = conf["credentials"]
      end

      # Authenticate a user with a given username and password
      # user - the username string
      # password - the password string
      # Returns true if credentials match, false otherwise
      def authenticate(user, password)
        # Check if the provided username exists in credentials and the password matches
        # Also ensure that user and password are not nil or false
        @good_credentials[user] == password && user && password
      end
    end
  end
end
```

lib/mlsmtp/authentication/login_handler.rb

```ruby
# Module defining SMTP server authentication logic
module SMTPServer
  # Nested module for authentication related methods
  module Authentication
    # Method handling LOGIN authentication process
    # Inputs:
    # - context: object containing connection context including methods to send and read data
    # - _: placeholder for additional arguments not used here
    def self.method_login_handler(context, _)
      # Log the start of LOGIN authentication attempt with verbosity level 5
      Logger.log "Trying auth LOGIN", origin: context.logger_origin, verbosity: 5

      # Create response requesting username in base64 encoding
      username_request = SMTP::Response.new(
        status: :positive_intermediate, # SMTP status indicating intermediate positive response
        category: :authentication,       # Response category
        detail: 4,                       # Detail code
        message: "VXNlcm5hbWU6"          # Base64 encoded message asking for username
      )
      # Send username request to client
      context.send_response(username_request)
      # Read base64 encoded username from client and decode it
      username = Base64.decode64(context.read[0])

      # Log the decoded username received from client
      Logger.log "Client username: #{username}", origin: context.logger_origin, verbosity: 5

      # Create response requesting password in base64 encoding
      password_request = SMTP::Response.new(
        status: :positive_intermediate, # SMTP status for intermediate response
        category: :authentication,
        detail: 4,
        message: "UGFzc3dvcmQ6"          # Base64 encoded message asking for password
      )
      # Send password request to client
      context.send_response(password_request)
```

```
      # Read base64 encoded password from client and decode it
      password = Base64.decode64(context.read[0])

      # Log that password has been received from client
      Logger.log "Client password received", origin: context.logger_origin, verbosity: 5

      # Check if the provided username and password are valid
      if Active.authenticate(username, password)
        # Create success response indicating authentication succeeded
        response = SMTP::Response.new(
          status: :positive_completed,     # SMTP status indicating completion
          category: :authentication,
          detail: 5,
          message: "2.7.0 Authentication Successful"
        )

        # Store the authenticated username in context
        context.authenticated_as = username

        # Log successful authentication
        Logger.log "Authenticated successfully", origin: context.logger_origin, verbosity: 5
      else
        # Create failure response indicating authentication failure
        response = SMTP::Response.new(
          status: :negative_permanent,       # SMTP status for permanent failure
          category: :authentication,
          detail: 5,
          message: "5.7.8 Authentication Failed"
        )

        # Log failed authentication attempt with warning level
        Logger.log "Authentication failed", origin: context.logger_origin, verbosity: 5, type: :warn
      end

      # Send the final authentication response to client
      context.send_response(response)
    end
  end
end
```

lib/mlsmtp/mail_lists/mail_list.rb

```
# Module SMTPServer encapsulates mail list functionality for SMTP server operations
module SMTPServer
  # Module MailLists groups classes related to mail list management
  module MailLists
    # Class MailList manages individual mail list objects and their database interactions
    class MailList
      # Initialize a mail list with a specific identifier
      # input: id - the unique identifier for the mail list
      def initialize(id)
        @id = id
      end

      # Expand method retrieves all email addresses associated with this mail list
      # returns: array of email addresses or nil if database is inactive
      def expand
        return nil unless Database.active # Check if database connection is active

        # Execute SQL query to select email addresses from list_memberships matching this list id
        Database.active.exec_sql(
          "SELECT email FROM list_memberships WHERE list_id = ?",
          @id
        ).map(&:first) # Map result to array of email strings
      end
```

```
end

# add_membership adds a new email to this mail list
# input: email - email address to add
# returns: true if successful, false if constraint exception occurs, nil if database inactive
def add_membership(email)
  return nil unless Database.active # Ensure database is active

  begin
    # Insert email and list id into list_memberships table
    Database.active.exec_sql(
      "INSERT INTO list_memberships (email, list_id) VALUES (?, ?)",
      [
        email,
        @id
      ]
    )
    return true # Return true if insert succeeds
  rescue SQLite3::ConstraintException
    return false # Return false if constraint exception occurs (e.g., duplicate)
  end
end

# remove_membership deletes an email from this mail list
# input: email - email address to remove
# returns: result of delete operation or nil if database inactive
def remove_membership(email)
  return nil unless Database.active # Check database connection

  # Execute delete SQL to remove matching email and list id from list_memberships
  Database.active.exec_sql(
    "DELETE FROM list_memberships WHERE (email = ? AND list_id = ?)",
    [
      email,
      @id
    ]
  )
end

# delete removes this mail list from the database
# returns: nil if database inactive
def delete
  return nil unless Database.active # Verify database connection

  # Execute delete SQL to remove mail list with this id from mail_lists table
  Database.active.exec_sql(
    "DELETE FROM mail_lists WHERE id = ?",
    [
      @id
    ]
  )
end

# Class method all retrieves all mail lists from database
# returns: array of mail list records or nil if database inactive
def self.all
  return nil unless Database.active # Check if database is active

  # Execute SQL to select all records from mail_lists table
  Database.active.exec_sql(
    "SELECT * FROM mail_lists"
  )
end

# Class method create adds a new mail list with specified name
# input: name - name of the new mail list
```

```
      # returns: MailList object if created or existing, nil if database inactive
      def self.create(name)
        return nil unless Database.active # Ensure database connection

        begin
          # Insert new mail list with provided name
          Database.active.exec_sql(
            "INSERT INTO mail_lists (name) VALUES (?)",
            [
              name
            ]
          )
        rescue SQLite3::ConstraintException
          # Ignore constraint exception (e.g., duplicate name)
          true
        end

        # Return MailList object found by the name
        find_by_name(name)
      end

      # Class method find_by_id retrieves a mail list by its id
      # input: id - identifier of the mail list
      # returns: MailList instance or nil if not found or database inactive
      def self.find_by_id(id)
        return nil unless Database.active # Check database connection

        # Execute SQL to select mail list with matching id
        list = Database.active.exec_sql(
          "SELECT * FROM mail_lists WHERE id = ?",
          [
            id,
          ]
        ).first

        # Instantiate MailList object if record exists, else return nil
        list ? new(list[0]) : nil
      end

      # Class method find_by_name retrieves a mail list by its name
      # input: name - name of the mail list
      # returns: MailList instance or nil if not found or database inactive
      def self.find_by_name(name)
        return nil unless Database.active # Check database connection

        # Execute SQL to select mail list with matching name, encoding to UTF-8
        list = Database.active.exec_sql(
          "SELECT * FROM mail_lists WHERE name = ?",
          [
            name.encode("UTF-8"),
          ]
        ).first

        # Instantiate MailList object if record exists, else return nil
        list ? new(list[0]) : nil
      end
    end
  end
end
```

lib/mlsmtp/patches/openssl_oid_safe_register.rb

```ruby
# Define a class within the OpenSSL::ASN1 module for handling Object Identifiers
class OpenSSL::ASN1::ObjectId
  # Save the original register method for later use
  @@original_register_method = self.method("register")

  # Override the register class method to add custom error handling
  # Inputs: variable number of arguments to be passed to the original register method
  def self.register(*args)
    begin
      # Call the original register method with provided arguments
      @@original_register_method.call(*args)
    rescue OpenSSL::ASN1::ASN1Error
      # If an ASN1Error occurs, return false indicating failure to register
      return false
    rescue => e
      # Re-raise any other exceptions encountered
      raise e
    end
  end
end
```

lib/mlsmtp/config.rb

```ruby
# This module defines a class for managing SMTP server configuration settings
module SMTPServer
  class Config
    # Class variable to hold list of required configuration setting keys
    @@required_conf_settings = []
    # Class variable to hold the currently active configuration instance
    @@active_config = nil

    # Initialize method for creating a configuration object
    # conf_settings is a hash of configuration key-value pairs
    def initialize(conf_settings = {})
      # Determine any missing required settings not provided in conf_settings
      missing_settings = @@required_conf_settings - conf_settings.keys
      # Raise an error if any required settings are missing
      unless missing_settings.empty?
        raise Errors::MissingConfigSettingError, missing_settings
      end
      # Store the provided configuration settings in an instance variable
      @settings = conf_settings
    end

    # Method to access a configuration value by key
    # k is the key to retrieve
    # Returns the value associated with the key as a string
    def [](k)
      @settings[k.to_s]
    end

    # Method to set a configuration value by key
    # k is the key, v is the value to assign
    def []=(k, v)
      @settings[k.to_s] = v
    end

    # Method to set this configuration as the active configuration
    def set_active
      @@active_config = self
    end

    # Class method to create a configuration object from a JSON string
    # text is the JSON string representing configuration settings
```

```
    # Returns a new Config object initialized with parsed JSON data
    def self.from_json(text)
      new(JSON.parse(text))
    end

    # Class method to create a configuration object from a file
    # filename is the path to the file containing JSON configuration data
    # Reads the file content and creates a Config object
    def self.from_file(filename)
      self.from_json(File.read(filename))
    end

    # Class method to access the list of required configuration settings
    def self.required_conf_settings
      @@required_conf_settings
    end

    # Class method to retrieve the current active configuration
    def self.active
      @@active_config
    end

    # Class method to clear the active configuration
    def self.clear_active
      @@active_config = {}
    end

    # Attribute accessor for the settings hash
    attr_accessor :settings
  end
end
```

lib/mlsmtp/storage/storage.rb

```
# Define a module SMTPServer with a nested module Storage
module SMTPServer
  module Storage
    # Retrieve the storage configuration from active configuration settings
    storage_config = Config.active["mail_storage"]["config"]
    # Retrieve the adapter name from active configuration settings
    adapter = Config.active["mail_storage"]["adapter"]
    # Instantiate a new object of the class specified by adapter with the storage configuration
    Active = Object.const_get(adapter).new(storage_config)
  end
end
```

lib/mlsmtp/storage/maildir.rb

```
module SMTPServer # Defines the main module for SMTP server related classes and modules
  module Storage # Defines a nested module for storage related classes
    class MaildirAdapter # Defines a class to adapt maildir storage for SMTP server
      # Initializes the adapter with settings including path and mailbox creation flag
      # settings - hash containing configuration options for the adapter
      def initialize(settings)
        @path = settings["path"] # Path to the maildir storage location
        @create_mailboxes = settings["create_mailboxes"] # Boolean flag to create mailboxes if they do not exist
      end

      # Adds a message to the specified user's mailbox
      # user - identifier for the mailbox owner
      # message - the email message to be stored
      def add_message(user, message)
        maildir(user).add(message) # Calls add method on Maildir object for the user
      end

      # Checks if a mailbox exists for the given user
      # user - identifier for the mailbox owner
      # Returns true if mailbox exists or if creation is allowed, false otherwise
      def mailbox_exists?(user)
        return false if user.to_s == "" # Return false if user string is empty
        File.exist?(maildir_path(user)) || @create_mailboxes # Check existence or create flag
      end

      # Initializes a mailbox for the user by creating a Maildir object
      # user - identifier for the mailbox owner
      # Returns the path to the mailbox directory
      def initialize_mailbox(user)
        md_path = maildir_path(user) # Get the path for the user's maildir
        Maildir.new(md_path) # Instantiate a new Maildir object at that path
        return md_path # Return the mailbox path
      end

      private

      # Retrieves a Maildir object for the user after verifying mailbox existence
      # user - identifier for the mailbox owner
      # Raises an error if mailbox does not exist
      # Returns a Maildir object for the user
      def maildir(user)
        raise Errors::NonexistentMailboxError, user unless mailbox_exists?(user) # Raise error if mailbox missing
        Maildir.new(maildir_path(user)) # Create and return Maildir object
      end

      # Constructs the path to the user's maildir by substituting special tokens
      # user - identifier for the mailbox owner
      # Returns the full path to the user's maildir
      def maildir_path(user)
        substitute_specials(@path, user: user) # Replace tokens in path string
      end

      # Replaces special placeholders in the path string with actual values
      # string - the path string containing placeholders
      # user - user identifier to substitute into the path
      # Returns the path string with placeholders replaced
      def substitute_specials(string, user:)
        string.gsub("%u", user) # Replace %u with the user identifier
      end
    end
  end
end
```

lib/mlsmtp/logger/logger.rb

```ruby
# MODULE SMTPServer contains the nested module Logger for logging functionalities
module SMTPServer
  module Logger
    # Define color mappings for different log message types
    @@colors = {
      info: :light_green,
      warn: :light_red,
      error: :red
    }

    # Load logger configuration settings from active configuration
    conf = Config.active["logger"]

    # Determine if logs should be output to standard output
    @@log_to_stdout = conf["log_to_stdout"]
    # Determine if logs should be written to files
    @@log_to_files = conf["log_to_files"]
    # Determine if colored output should be used in stdout
    @@stdout_colors = conf["stdout_colors"]
    # Set the format for timestamps in log messages
    @@strftime = conf["time_format"]
    # Set the maximum verbosity level for logging
    @@max_verbosity_level = conf["max_verbosity_level"]

    # Method to log messages with optional origin, type, and verbosity
    # Inputs:
    # - message: the log message string
    # - origin: string indicating the source of the log, default "System"
    # - type: symbol indicating log type, default :info (e.g., :warn, :error)
    # - verbosity: integer indicating verbosity level, default 0
    def self.log(message, origin: "System", type: :info, verbosity: 0)
      # Check if message verbosity is within the maximum allowed level
      unless verbosity <= @@max_verbosity_level || @@max_verbosity_level < 0
        return false # Do not log if verbosity exceeds maximum
      end

      # Format the log line with timestamp, type, origin, and message
      line = format_line(message, origin: origin, type: type)

      # Output to stdout if enabled
      if @@log_to_stdout
        # Use color if enabled, otherwise print plain line
        STDOUT.puts(
          @@stdout_colors ? line.color(@@colors[type]) : line
        )
      end

      # Write the log line to each configured log file
      @@log_to_files.each do |file|
        File.write(file, "#{line}\n", mode: ?a) # Append mode
      end
    end

    # Helper method to format a log line with timestamp, type, origin, and message
    # Inputs:
    # - message: the log message string
    # - origin: string indicating source of log
    # - type: symbol indicating log type
    # Output:
    # - formatted string with timestamp, type, origin, and message
    def self.format_line(message, origin:, type:)
      # Generate timestamp string based on configured format
      time_text = "[#{Time.now.strftime(@@strftime)}]"
      # Convert type symbol to uppercase string within brackets
      type_text = "[#{type.to_s.upcase}]"
```

```
      # Include origin within brackets
      origin_text = "[#{origin}]"

      # Concatenate all parts into a single log line string
      return "#{time_text} #{type_text} #{origin_text}: #{message}"
    end
  end
end
```

lib/mlsmtp/ssl/certificates.rb

```
# Define the SMTPServer module with nested SSL module and Certificate class
module SMTPServer
  module SSL
    class Certificate
      # Class variable to store all certificate instances by name
      @@certificates = {}

      # Initialize method to create a new certificate object
      # Inputs:
      # - name: optional string name for the certificate, defaults to "cert"
      # - certificate: optional string or OpenSSL certificate object
      # - key: string or OpenSSL key object, required
      def initialize(name="cert", certificate: nil, key:)
        @name = name

        # Convert key input to OpenSSL::PKey::RSA if it's a string
        @key = key.is_a?(String) ? OpenSSL::PKey::RSA.new(key) : key

        if certificate
          # Convert certificate input to OpenSSL::X509::Certificate if it's a string
          @certificate = certificate.is_a?(String) ? OpenSSL::X509::Certificate.new(certificate) : certificate

          # Create a new SSL context and assign certificate and key
          @context = OpenSSL::SSL::SSLContext.new
          @context.cert = @certificate
          @context.key = @key
        end

        # Store the created certificate instance in the class variable hash
        @@certificates[@name] = self
      end

      # Method to delete the certificate instance from the class variable hash
      def destroy
        @@certificates.delete(@name)
      end

      # Class method to retrieve a certificate instance by name
      # Input:
      # - k: string name of the certificate
      # Output:
      # - certificate instance or nil if not found
      def self.[](k)
        @@certificates[k]
      end

      # Class method to assign a certificate instance to a name
      # Inputs:
      # - k: string name
      # - v: certificate instance
      def self.[]=(k, v)
        @@certificates[k] = v
      end
```

```
      # Class method to retrieve all stored certificates
      # Output:
      # - hash of all certificate instances keyed by name
      def self.all
        @@certificates
      end

      # Accessor methods for certificate, key, and context
      attr_accessor :certificate, :key, :context
      # Read-only attribute for name
      attr_reader :name

      # Load certificate configurations from application settings
      # For each configuration, read certificate and key files and create a new certificate instance
      Config.active["certificates"].each do |name, info|
        # Read certificate file if cert_path is provided
        cert = info["cert_path"] ? File.read(info["cert_path"]) : nil
        # Read key file
        key = File.read(info["key_path"])

        # Create new certificate instance with loaded data
        new(name, certificate: cert, key: key)
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_command_handlers/starttls.rb

```
module SMTPServer
  # Defines the SMTPServer module as a namespace for server-related classes and modules
  module Handlers
    # Defines the Handlers module to group command handler methods
    module SMTPServerCommandHandlers
      # Defines a class method starttls that handles the STARTTLS command
      # Inputs:
      #   - context: an object representing the current connection state and methods
      def self.starttls(context)
        # Logs an attempt to upgrade the connection to TLS with verbosity level 5
        Logger.log "Attempting to upgrade to TLS", origin: context.logger_origin, verbosity: 5

        # Creates a new SMTP command object for STARTTLS
        command = SMTP::Command.new("STARTTLS")
        # Sends the STARTTLS command to the server
        context.send_data(command)

        # Reads the server's response to the STARTTLS command
        response = context.read_response

        # Checks if the server's response code is not 220 indicating failure
        if response.code != "220"
          # Logs a warning that the server rejected the STARTTLS request with the given code
          Logger.log "Server rejected STARTTLS request with code `#{response.code}`", origin: context.logger_origin, verbosity: 5, t
          # Sets the connection state to be ready for MAIL FROM command after rejection
          context.ready_for = :mailfrom
          return
        end

        begin
          # Creates a new SSL context with default settings
          ssl_context = OpenSSL::SSL::SSLContext.new
          # Sets the SSL context to not verify server certificates (for simplicity or testing)
          ssl_context.verify_mode = OpenSSL::SSL::VERIFY_NONE
          # Wraps the existing TCP socket with an SSL socket for secure communication
          ssl_socket = OpenSSL::SSL::SSLSocket.new(context.tcp_server, ssl_context)
          # Ensures the socket closes properly when done
          ssl_socket.sync_close = true
          # Initiates the SSL/TLS handshake
          ssl_socket.connect

          # Replaces the current connection in context with the SSL socket
          context.server = ssl_socket

          # Logs that the connection has been successfully upgraded to TLS
          Logger.log "Upgraded to TLS", origin: context.logger_origin, verbosity: 5

          # Sets the connection state to be ready for the extended HELO/EHLO command
          context.ready_for = :re_ehlo
        rescue OpenSSL::SSL::SSLError
          # Handles SSL handshake failures by logging a warning
          Logger.log "Failed upgrading to TLS", origin: context.logger_origin, verbosity: 5, type: :warn

          # Resets the connection state to be ready for MAIL FROM after failure
          context.ready_for = :mailfrom
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_command_handlers/quit.rb

```
 # Define the module hierarchy for SMTP server command handlers
module SMTPServer
  module Handlers
    module SMTPServerCommandHandlers

      # This method handles the QUIT command for the SMTP server
      # Inputs:
      # - context: the connection context object that manages communication
      # - negative: optional boolean indicating if the quit is due to an error, defaults to false
      def self.quit(context, negative: false)
        # Create a new SMTP command object with the command name QUIT
        command = SMTP::Command.new("QUIT")
        # Send the QUIT command data to the client through the context
        context.send_data(command)
        # Close the connection to the client
        context.close

        # Log the outcome of the connection closure
        # Log as warning if negative is true, otherwise as info
        Logger.log "Closed connection to server with #{negative ? "failure" : "success"}",
                   origin: context.logger_origin,
                   verbosity: 5,
                   type: negative ? :warn : :info
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_command_handlers/ehlo.rb

```
module SMTPServer
  module Handlers
    module SMTPServerCommandHandlers
      # This method performs the SMTP EHLO handshake with the server.
      # Inputs:
      #   - context: the connection context object managing communication
      #   - attempt_starttls: boolean flag to decide if STARTTLS should be attempted
      # Output:
      #   - returns true if EHLO is accepted, false otherwise
      def self.ehlo(context, attempt_starttls: true)
        # Retrieve the mail server name from configuration
        heloname = Config.active["mailname"]
        # Create an SMTP command object with EHLO and the server name
        command = SMTP::Command.new("EHLO", heloname)
        # Send the EHLO command to the server
        context.send_data(command)
        # Log the identification attempt with the server name
        Logger.log "Self-Identified as `#{heloname}` with EHLO", origin: context.logger_origin, verbosity: 5
        # Read the server's response to the EHLO command
        response = context.read_response
        # Check if the server accepted the EHLO command (response code starting with 2)
        if response.status == 2
          # Log successful EHLO acceptance
          Logger.log "Server accepted EHLO", origin: context.logger_origin, verbosity: 5
          # Check if the server's response includes STARTTLS capability and if we want to attempt it
          if response.message.include?("STARTTLS") && attempt_starttls
            # Prepare to initiate STARTTLS handshake
            context.ready_for = :starttls
          else
            # Proceed to MAIL FROM command after successful EHLO
            context.ready_for = :mailfrom
          end
          # Return true indicating EHLO was successful
          return true
        else
          # Log rejection of EHLO with server's response code
          Logger.log "Server rejected EHLO with code `#{response.code}, trying HELO`", origin: context.logger_origin, verbosity: 5,
          # Fallback to HELO command if EHLO is rejected
          context.ready_for = :helo
          # Return false indicating EHLO failed
          return false
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_command_handlers/rcptto.rb

```ruby
# Define the SMTPServer module with nested modules for organization
module SMTPServer
  module Handlers
    module SMTPServerCommandHandlers
      # This method handles the SMTP RCPT TO command to specify the recipient address
      # Inputs:
      #   context - an object containing the current SMTP session state and methods
      # Returns:
      #   true if the server accepts the recipient, false otherwise
      def self.rcptto(context)
        # Format the recipient address within angle brackets
        rcpt_to = "<#{context.recipient_addr}>"
        # Create a new SMTP command object with the RCPT TO command and recipient address
        command = SMTP::Command.new("RCPT TO:", rcpt_to)

        # Send the command to the SMTP server
        context.send_data(command)

        # Log the recipient address being identified
        Logger.log "Identified recipient as #{rcpt_to}", origin: context.logger_origin, verbosity: 5

        # Read the server's response to the RCPT TO command
        response = context.read_response

        # Check if the server's response indicates success (status code 2)
        if response.status == 2
          # Log acceptance of the recipient address
          Logger.log "Server accepted recipient address", origin: context.logger_origin, verbosity: 5
          # Update the context to be ready for the data phase
          context.ready_for = :data
          # Indicate success
          return true
        else
          # Log the unexpected server response with warning level
          Logger.log "Unexpected response `#{response.code}`", origin: context.logger_origin, verbosity: 5, type: :warn
          # Update the context to indicate a negative quit state
          context.ready_for = :quit_negative
          # Indicate failure
          return false
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_command_handlers/helo.rb

```
# Define the SMTPServer module containing nested modules for handlers and command handlers
module SMTPServer
  module Handlers
    module SMTPServerCommandHandlers
      # This method handles the SMTP HELO command
      # It takes a context object as input and returns true if the server accepts the HELO
      def self.helo(context)
        # Retrieve the active mail hostname from the configuration
        heloname = Config.active["mailname"]
        # Create a new SMTP command object with HELO and the hostname
        command = SMTP::Command.new("HELO", heloname)

        # Send the HELO command to the server via the context
        context.send_data(command)

        # Log the identification attempt with the hostname
        Logger.log "Self-Identified as `#{heloname}` with HELO", origin: context.logger_origin, verbosity: 5

        # Read the server's response to the HELO command
        response = context.read_response

        # Check if the response status code indicates success (2xx)
        if response.status == 2
          # Log acceptance of the HELO command
          Logger.log "Server accepted HELO", origin: context.logger_origin, verbosity: 5
          # Update context state to be ready for MAIL FROM command
          context.ready_for = :mailfrom
          # Return true indicating successful HELO handshake
          return true
        else
          # Log a warning if the server's response was not as expected
          Logger.log "Unexpected HELO response `#{response.code}`", origin: context.logger_origin, verbosity: 5, type: :warn
          # Update context state to be ready for quit or negative response
          context.ready_for = :quit_negative
          # Return false indicating failure in HELO handshake
          return false
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_command_handlers/data.rb

```
# Module defining SMTP server command handlers within the SMTPServer namespace
module SMTPServer
  module Handlers
    module SMTPServerCommandHandlers
      # Method to handle the SMTP DATA command
      # Input: context object that manages communication and state
      # Output: boolean indicating success or failure of sending data
      def self.data(context)
        # Create a new SMTP command object with the command name DATA
        command = SMTP::Command.new("DATA")
        # Send the DATA command to the SMTP client/server
        context.send_data(command)
        # Log that a data send request has been made with a verbosity level of 5
        Logger.log "Requested to send data", origin: context.logger_origin, verbosity: 5
        # Read the response from the SMTP server after sending DATA command
        response = context.read_response
        # Check if the response code indicates readiness to receive data (354)
        if response.code == "354"
          # Log that data transmission is starting
          Logger.log "Sending data", origin: context.logger_origin, verbosity: 5
          # Retrieve the data to be sent from the context
          data = context.data
          # If data is a string, split it into lines based on CRLF or LF
          data = data.split(/\r?\n/) if data.is_a?(String)
          # Iterate over each line in the data
          data.each do |line|
            # If a line starts with a dot, prepend another dot to escape it
            line = ".#{line}" if line[0] == ?.
            # Send the (possibly escaped) line to the SMTP server
            context.send_data(line)
          end
          # Send a line with only a dot to indicate end of data transmission
          context.send_data(?.)
          # Read the server's response after data transmission
          response = context.read_response
          # Check if the response status indicates success (status code starting with 2)
          if response.status == 2
            # Set the next expected state to quit positively
            context.ready_for = :quit_positive
            # Return true indicating successful data transmission
            return true
          else
            # Set the next expected state to quit negatively
            context.ready_for = :quit_negative
            # Return false indicating failure in data transmission
            return false
          end
        else
          # Log a warning if the response code was not 354
          Logger.log "Unexpected response `#{response.code}`", origin: context.logger_origin, verbosity: 5, type: :warn
          # Set the next state to quit negatively due to unexpected response
          context.ready_for = :quit_negative
          # Return false indicating failure to proceed with data transmission
          return false
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_command_handlers/mailfrom.rb

```
# Define the SMTPServer module containing nested modules for handling SMTP server commands
module SMTPServer
  module Handlers
    module SMTPServerCommandHandlers
      # This method processes the MAIL FROM command in SMTP protocol
      # It takes a context object as input and returns true if the sender address is accepted, false otherwise
      def self.mailfrom(context)
        # Format the sender address by wrapping the sender address in angle brackets
        mail_from = "<#{context.sender_addr}>"
        # Create a new SMTP command object with command name MAIL FROM and the formatted sender address
        command = SMTP::Command.new("MAIL FROM:", mail_from)

        # Send the command to the SMTP server via the context's send_data method
        context.send_data(command)

        # Log the identified sender address with verbosity level 5
        Logger.log "Identified sender as #{mail_from}", origin: context.logger_origin, verbosity: 5

        # Read the server's response to the MAIL FROM command
        response = context.read_response

        # Check if the server's response status code indicates success (status code 2)
        if response.status == 2
          # Log acceptance of sender address
          Logger.log "Server accepted sender address", origin: context.logger_origin, verbosity: 5
          # Set the next expected command to RCPT TO
          context.ready_for = :rcptto
          # Return true indicating success
          return true
        else
          # Log a warning with the unexpected response code
          Logger.log "Unexpected response `#{response.code}`", origin: context.logger_origin, verbosity: 5, type: :warn
          # Set the next state to quit negative due to failure
          context.ready_for = :quit_negative
          # Return false indicating failure
          return false
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/generic_client_handler.rb

```
# This module defines an SMTP server handler for a generic SMTP client session.
module SMTPServer
  module Handlers
    # This class manages the communication with a SMTP client.
    class GenericClientHandler
      # Initializes the handler with a given context object.
      # Inputs:
      # - context: an object containing session state and methods for communication.
      def initialize(context)
        @context = context
      end

      # Handles the SMTP client session.
      # Sends banner if not already sent, then processes commands until connection closes.
      def handle_client
        # Send server banner if it hasn't been sent yet.
        unless @context.banner_sent
          @context.send_banner # Send greeting banner to client.
          Logger.log "Sending banner to client", origin: @context.logger_origin, verbosity: 4
        end
```

```
# Loop until the connection is closed.
until @context.closed
  raw_command = @context.read[0] # Read raw command from client.
  begin
    # Parse the raw command into a command object.
    command = SMTP::Command.parse(raw_command)
  rescue SMTPServer::Errors::InvalidCommandError => e
    # Log invalid command error.
    Logger.log "Invalid command from client: `#{e.command}`", type: :warn, origin: @context.logger_origin, verbosity: 4
    # Create a response indicating command not recognized.
    response = SMTP::Response.new(
      status: :negative_permanent, # Permanent negative response.
      category: :syntax,
      detail: 2,
      message: "5.5.2 command not recognized"
    )
    # Send error response to client.
    @context.send_response(response)
    next # Continue to next command.
  rescue SMTPServer::Errors::IncompleteCommandError => e
    # Log incomplete command error.
    Logger.log "Incomplete command from client: `#{e.original_command}`", type: :warn, origin: @context.logger_origin, verbo
    # Prepare syntax error response with expected command format.
    response = SMTP::Response.new(
      status: :negative_permanent,
      category: :syntax,
      detail: 1,
      message: "5.5.4 Syntax: #{e.template_command.map{|x| x == String ? "<argument>" : x}.join(" ")}"
    )
    # Send syntax error response.
    @context.send_response(response)
    next # Continue to next command.
  end

  # Handle the command based on its command keyword.
  case command.command
  # ESMTP commands
  when [ "EHLO" ]
    SMTPCommandHandlers.ehlo(@context, command.values)
  when [ "STARTTLS" ]
    SMTPCommandHandlers.starttls(@context)
  when [ "AUTH" ]
    SMTPCommandHandlers.auth(@context, command.values)

  # SMTP commands
  when [ "HELO" ]
    SMTPCommandHandlers.helo(@context, command.values)
  when [ "MAIL", "FROM:" ]
    SMTPCommandHandlers.mailfrom(@context, command.values)
  when [ "RCPT", "TO:" ]
    SMTPCommandHandlers.rcptto(@context, command.values)
  when [ "DATA" ]
    SMTPCommandHandlers.data(@context)
  when [ "QUIT" ]
    SMTPCommandHandlers.quit(@context)
  when [ "RSET" ]
    SMTPCommandHandlers.rset(@context)
  when [ "VRFY" ]
    SMTPCommandHandlers.vrfy(@context, command.values)
  when [ "EXPN" ]
    SMTPCommandHandlers.expn(@context, command.values)
  when [ "NOOP" ]
    SMTPCommandHandlers.noop(@context)
  end
end
```

```
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/starttls.rb

```
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # This method handles the STARTTLS command in SMTP
      # Input: context - an object representing the current SMTP session state
      def self.starttls(context)
        # Check if the current context supports STARTTLS
        unless context.starttls_support
          # Create a negative response indicating command not recognized
          response = SMTP::Response.new(
            status: :negative_permanent, # SMTP permanent negative response
            category: :syntax,           # Response category indicating syntax error
            detail: 2,                   # Specific detail code for command not supported
            message: "5.5.2 Error: command not recognized" # Error message
          )
          # Send the response back to the client
          context.send_response(response)

          # Log the event that STARTTLS is not supported in this context
          Logger.log "This context does not support STARTTLS", origin: context.logger_origin, verbosity: 5, type: :warn

          # Exit the method early since STARTTLS cannot be initiated
          return
        end

        # Check if the connection is already using STARTTLS
        if context.using_starttls
          # Create a negative response indicating STARTTLS is already in use
          response = SMTP::Response.new(
            status: :negative_permanent, # SMTP permanent negative response
            category: :syntax,           # Response category indicating syntax error
            detail: 3,                   # Specific detail code for already using STARTTLS
            message: "5.5.2 Error: already using STARTTLS" # Error message
          )
          # Send the response back to the client
          context.send_response(response)

          # Log that STARTTLS was attempted but already active
          Logger.log "Already using STARTTLS", origin: context.logger_origin, verbosity: 5, type: :warn

          # Exit early since STARTTLS is already active
          return
        end

        # Create a positive response indicating readiness to start TLS
        response = SMTP::Response.new(
          status: :positive_completed, # SMTP positive completion status
          category: :connections,      # Category indicating connection-related response
          message: "2.0.0 Ready to start TLS" # Success message
        )

        # Send the readiness response to the client
        context.send_response(response)

        begin
          # Retrieve the SSL context (certificate and key) for STARTTLS
```

```
          ssl_context = context.starttls_certificate
          # Create a new SSL socket wrapping the existing TCP connection
          ssl_socket = OpenSSL::SSL::SSLSocket.new(context.tcp_client, ssl_context)
          ssl_socket.sync_close = true # Ensure socket closes properly
          ssl_socket.accept            # Perform SSL handshake

          # Replace the current client socket with the SSL socket
          context.client = ssl_socket

          # Mark the context as now using STARTTLS
          context.using_starttls = true

          # Log the successful upgrade to TLS
          Logger.log "Upgraded to TLS", origin: context.logger_origin, verbosity: 5
        rescue OpenSSL::SSL::SSLError
          # Log a warning if SSL handshake fails
          Logger.log "Failed upgrading to TLS", origin: context.logger_origin, verbosity: 5, type: :warn
        end
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/auth.rb

```
 # This module defines handlers for SMTP commands within a server implementation.
# Specifically, it includes a method to process the SMTP AUTH command for authentication.

module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # This method processes the SMTP AUTH command.
      # Inputs:
      # - context: the current connection context which includes authentication state and methods.
      # - args: array of arguments passed with the AUTH command, where args[0] is the authentication method.
      def self.auth(context, args)
        # Retrieve the list of valid authentication methods from configuration.
        auth_methods = Config.active["authentication"]["valid_auth_methods"].keys

        # Check if the client is already authenticated.
        if context.authenticated_as
          # Create a response indicating that re-authentication is not allowed.
          message = SMTP::Response.new(
            status: :negative_permanent,   # Permanent failure status.
            category: :syntax,              # Response category indicating syntax error.
            detail: 3,                      # Specific detail code.
            message: "5.5.1 Error: repeated authentication" # Error message.
          )
          # Send the error response to the client.
          context.send_response(message)
          return
        end

        # Validate the requested authentication method against allowed methods.
        unless auth_methods.include?(args[0].upcase)
          # Create a response indicating invalid authentication mechanism.
          response = SMTP::Response.new(
            status: :negative_permanent,   # Permanent failure.
            category: :authentication,      # Authentication error category.
            detail: 5,                      # Specific error detail.
            message: "5.7.8 Invalid Mechanism" # Error message.
          )
          # Send the invalid mechanism response.
          context.send_response(response)

          # Log a warning about the invalid authentication attempt.
          Logger.log "Client requested bad authentication mechanism #{args[0].upcase}", origin: context.logger_origin, verbosity: 5,
          return
        end

        # Retrieve the class and method for the specified authentication mechanism.
        auth_const, auth_method = Config.active["authentication"]["valid_auth_methods"][args[0].upcase]

        # Dynamically invoke the authentication method with the context and remaining arguments.
        Object.const_get(auth_const).method(auth_method).call(context, args[1..-1])
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/quit.rb

```
# Define the SMTPServer module which contains nested modules for organization
module SMTPServer
  module Handlers
    module SMTPCommandHandlers

      # The quit method handles the SMTP QUIT command from a client
      # It takes a context object as input which manages connection state
      def self.quit(context)
        # Create a response indicating successful connection closure
        response = SMTP::Response.new(
          status: :positive_completed,  # Response status indicating success
          category: :connections,        # Category of response related to connection management
          detail: 1,                     # Detail code for the response
          message: "2.0.0 Closing connection"  # Human-readable message
        )
        # Send the response back to the client
        context.send_response(response)
        # Close the client's connection
        context.close

        # Log the disconnection event with a specific verbosity level
        Logger.log "Client disconnected with QUIT command", origin: context.logger_origin, verbosity: 5
      end

    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/ehlo.rb

```
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # This method handles the SMTP EHLO command
      # Inputs:
      # - context: the current connection context object
      # - args: array of arguments provided with the EHLO command
      def self.ehlo(context, args)
        # Set the hostname from the first argument
        context.heloname = args[0]
        # Enable Extended SMTP features
        context.esmtp = true
        # Initialize mailfrom state to ready if not already set
        context.mailfrom = :ready unless context.mailfrom

        # Retrieve list of valid authentication methods from configuration
        auth_methods = Config.active["authentication"]["valid_auth_methods"].keys

        # Build the initial list of SMTP response messages
        esmtp_message = [
          Config.active["mailname"], # Server's mail name
          "SIZE #{Config.active["max_size"]}", # Max message size supported
          "PIPELINING", # Support for pipelining
          "ENHANCEDSTATUSCODES", # Support for enhanced status codes
        ]
        # Add 8BITMIME support if enabled in configuration
        esmtp_message += [ "8BITMIME", "SMTPUTF8" ] if Config.active["support_8_bit"]
        # Add VRFY command unless it is disabled in configuration
        esmtp_message << "VRFY" unless Config.active["disable_commands"].include?("VRFY")
        # Add STARTTLS command if the connection supports TLS
        esmtp_message << "STARTTLS" if context.starttls_support
        # Add AUTH command with supported methods if any are available
        esmtp_message << "AUTH #{auth_methods.join(" ")}" unless auth_methods.empty?
        # Alternative syntax for AUTH command
        esmtp_message << "AUTH=#{auth_methods.join(" ")}" unless auth_methods.empty?

        # Create a response object with a positive completion status
        response = SMTP::Response.new(
          status: :positive_completed, # SMTP success status
          category: :mail_system, # Category of the response
          message: esmtp_message # List of SMTP extension features
        )

        # Send the constructed response back to the client
        context.send_response(response)

        # Log the EHLO command received from the client
        Logger.log "Client EHLO: #{context.heloname}", origin: context.logger_origin, verbosity: 5
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/vrfy.rb

```
# Define the module SMTPServer which contains nested modules for handling SMTP commands
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # This method handles the VRFY SMTP command which verifies if an email address exists
      # Inputs:
      # - context: the current server context which includes methods for sending responses and logging
      # - args: an array of arguments passed with the VRFY command, where args[0] is the email address to verify
      def self.vrfy(context, args)
        email = args[0] # Extract the email address from arguments

        # Create a destination object for the email address, which determines if it's local
        destination = Transport::Destination.new(email)

        # Check if the destination is local and if the mailbox exists for the user
        if destination.local && Storage::Active.mailbox_exists?(destination.destination_user)
          # If mailbox exists, create a positive response indicating success
          response = SMTP::Response.new(
            status: :positive_completed, # SMTP status code indicating success
            category: :mail_system,       # Category of the response
            message: email                # Message containing the verified email address
          )

          # Log the successful verification with verbosity level 5
          Logger.log "Requested address `#{email}` exists on this server", origin: context.logger_origin, verbosity: 5
        else
          # If mailbox does not exist, create a negative permanent failure response
          response = SMTP::Response.new(
            status: :negative_permanent,  # SMTP status code indicating permanent failure
            category: :mail_system,       # Category of the response
            message: "User does not exist" # Message indicating user not found
          )

          # Log the failed verification attempt with verbosity level 5
          Logger.log "Requested address `#{email}` does not exist on this server", origin: context.logger_origin, verbosity: 5
        end

        # Send the constructed response back to the client through the context
        context.send_response(response)
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/rcptto.rb

```
# Define the SMTPServer module containing nested modules for handling SMTP commands
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # This method processes the RCPT TO command with given context and arguments
      # Inputs:
      #   - context: object holding the current connection state and methods
      #   - args: array of arguments provided with the command, expecting recipient email
      def self.rcptto(context, args)
        # Check if the context has already received a MAIL FROM command
        unless context.rcptto
          # Log a warning if RCPT TO is received before MAIL FROM
          Logger.log "Received unexpected RCPT TO: command", origin: context.logger_origin, verbosity: 5, type: :warn

          # Create a response indicating a syntax error due to missing MAIL FROM
          message = SMTP::Response.new(
            status: :negative_permanent,   # Permanent negative response
            category: :syntax,              # Syntax error category
            detail: 3,                      # Detail code (could be application-specific)
            message: "5.5.1 Error: MAIL FROM: required"  # Error message
          )
          # Send the error response back to the client
          context.send_response(message)
          # Exit the method early due to error
          return
        end

        # Log the recipient email address being added
        Logger.log "Recipient added: `#{args[0]}`", origin: context.logger_origin, verbosity: 5

        # Append the recipient address to the context's recipient list
        context.rcptto << args[0]

        # Instantiate an authorization handler with the current context
        auth_handler = Transport::AuthorizationHandler.new(context)
        # Perform authorization handling for the current recipient
        auth_handler.handle_authorization
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/noop.rb

```
# Define the SMTPServer module which contains the Handlers module
module SMTPServer
  module Handlers
    # Define the SMTPCommandHandlers module which contains command handling methods
    module SMTPCommandHandlers
      # Defines a class method noop that handles the NOOP SMTP command
      # Input: context - an object representing the current SMTP session or connection
      def self.noop(context)
        # Create a new SMTP response object with a positive completion status
        response = SMTP::Response.new(
          status: :positive_completed,    # Status indicating command was successful
          category: :mail_system,        # Category of the response
          detail: 0,                     # Detail code for the response
          message: "2.0.0 Ok"            # Human-readable message for the client
        )
        # Send the constructed response back to the client via the context object
        context.send_response(response)
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/rset.rb

```
# Define the SMTPServer module which contains nested modules for organizing SMTP handling code
module SMTPServer
  module Handlers
    module SMTPCommandHandlers

      # Define a class method rset that handles the SMTP RSET command
      # It takes a context object as input which maintains the connection state
      def self.rset(context)
        # Reset the current session or connection state within the context
        context.reset

        # Create a new SMTP response object indicating successful reset
        response = SMTP::Response.new(
          status: :positive_completed,  # Response status indicating success
          category: :mail_system,      # Category of the response
          detail: 0,                   # Additional detail code
          message: "2.0.0 Ok"          # Human-readable message
        )

        # Send the constructed response back to the SMTP client
        context.send_response(response)

        # Log the reset action with a specified verbosity level
        Logger.log "Reset client", origin: context.logger_origin, verbosity: 5
      end

    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/expn.rb

```ruby
# Module defining SMTP command handlers within the SMTP server
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # Method to handle the EXPN SMTP command
      # Inputs:
      # - context: object containing connection and logger info
      # - args: array of arguments provided with the EXPN command
      def self.expn(context, args)
        # Create a destination object with the first argument and get its address
        list = Transport::Destination.new(args[0], get_servers: false).address

        # Find the mailing list by its name and expand it into its members
        list_members = MailLists::MailList.find_by_name(list)&.expand

        # Check if the list exists and has members
        if list_members.nil? || list_members.empty?
          # Prepare a negative response indicating the mailing list is empty or does not exist
          response = SMTP::Response.new(
            status: :negative_permanent, # Permanent failure status
            category: :mail_system,
            message: "Mailing list is empty or does not exist"
          )

          # Log the failed attempt to expand the mailing list
          Logger.log "Requested email list `#{list}` is empty or does not exist", origin: context.logger_origin, verbosity: 5, type:
        else
          # Prepare a positive response with the list members formatted as email addresses
          response = SMTP::Response.new(
            status: :positive_completed, # Successful completion status
            category: :mail_system,
            message: list_members.map{|m| "<#{m}>"}
          )

          # Log the successful expansion of the mailing list
          Logger.log "Returned #{list_members.length} members of list #{list}", origin: context.logger_origin, verbosity: 5
        end

        # Send the response back to the SMTP client
        context.send_response(response)
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/helo.rb

```
# Define a module hierarchy for SMTP command handlers within the SMTPServer module
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # Define a class method to handle the HELO SMTP command
      # Inputs:
      # - context: an object representing the current SMTP session state
      # - args: an array of arguments passed with the HELO command, typically the hostname
      def self.helo(context, args)
        # Set the session's heloname to the first argument provided
        context.heloname = args[0]
        # Initialize mailfrom attribute to ready state if it is not already set
        context.mailfrom = :ready unless context.mailfrom

        # Create a new SMTP response object with positive completion status
        response = SMTP::Response.new(
          status: :positive_completed,         # Indicate command was successful
          category: :mail_system,               # Categorize the response under mail system
          message: Config.active["mailname"]   # Use configured mail system name as message
        )

        # Send the constructed response back to the SMTP client
        context.send_response(response)

        # Log the HELO command received from the client with verbosity level 5
        Logger.log "Client HELO: #{context.heloname}", origin: context.logger_origin, verbosity: 5
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/data.rb

```
# This module defines handlers for SMTP commands within the SMTPServer namespace
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # This method handles the SMTP DATA command
      # Input: context - an object representing the current SMTP session state
      def self.data(context)
        # Check if the server is ready to receive data
        unless context.data == :ready
          # Log unexpected DATA command reception
          Logger.log "Received unexpected DATA command", origin: context.logger_origin, verbosity: 5, type: :warn

          # Create a negative response indicating MAIL FROM is required before DATA
          message = SMTP::Response.new(
            status: :negative_permanent,
            category: :syntax,
            detail: 3,
            message: "5.5.1 Error: MAIL FROM: required"
          )
          # Send the error response to the client
          context.send_response(message)
          return
        end

        # Log that DATA command was received and server is awaiting message data
        Logger.log "Received DATA command, waiting for message", origin: context.logger_origin, verbosity: 5

        # Send positive intermediate response indicating end of data with CRLF dot
        message = SMTP::Response.new(
          status: :positive_intermediate,
          category: :mail_system,
```

```
      detail: 4,
      message: "End data with <CRLF>.<CRLF>"
    )
    context.send_response(message)

    # Read message data until a line containing only a dot
    data = context.read(read_until: ".").map{ |l| l[0] == ?. ? l[1..-1] : l }.join("\r\n")
    # Remove leading dot from lines that start with a dot as per SMTP transparency rules

    # Check if message size exceeds maximum allowed size
    if data.length > Config.active["max_size"]
      # Create a permanent failure response indicating message size is too large
      message = SMTP::Response.new(
        status: :negative_permanent,
        category: :mail_system,
        detail: 2,
        message: "5.3.4 Message exceeds max size"
      )
      # Send the size error response
      context.send_response(message)

      # Log that message exceeded size limit
      Logger.log "Message exceeds max size", origin: context.logger_origin, verbosity: 5, type: :warn
      return
    end

    # Store the received data in the context object
    context.data = data

    # Check if authorization is required for processing post data
    unless context.authorization_exempt
      # Iterate over configured authorization adapters
      Config.active["postdata_authorization_adapters"].each do |adapter, config|
        # Split adapter string into class and method names
        auth_const, auth_method = adapter.split(?#)

        # Call the authorization method dynamically
        unless Object.const_get(auth_const).method(auth_method).call(context, config)
          # Create a permanent failure response for authorization failure
          message = SMTP::Response.new(
            status: :negative_permanent,
            category: :authentication,
            detail: 5,
            message: "5.7.8 Error: authorization failed"
          )
          # Send authorization failure response
          context.send_response(message)
          # Reset session state after failure
          context.reset

          # Log authorization failure
          Logger.log "Authorization failed; rejecting message", origin: context.logger_origin, verbosity: 5, type: :warn
          return
        end
      end
    else
      # Log skipping authorization due to exemption
      Logger.log "Authorization exempt; skipping authorization adapters", origin: context.logger_origin, verbosity: 5
    end

    # Initialize array to hold queue IDs for messages
    queue_ids = []

    # Prepare email message with headers
    preparer = Email::EmailPreparer.new(context)
    preparer.add_all_headers
```

```
        preparer.add_all_headers

        i = 0
        # Loop through recipient addresses
        while i < context.rcptto.length
          rcpt_to = context.rcptto[i]
          i += 1

          # Convert recipient address to fixed destination object
          fixed_address = Transport::Destination.new(rcpt_to, get_servers: false).address

          # Check if recipient is a mailing list
          mail_list = MailLists::MailList.find_by_name(fixed_address)

          if mail_list
            # Expand mailing list recipients
            context.rcptto.delete(rcpt_to)
            context.rcptto += mail_list.expand
            i -= 1
            next
          end

          # Create queued message object for each recipient
          message = Queue::QueuedMessage.new(
            mail_from: Transport::Destination.new(context.mailfrom, get_servers: false).address,
            rcpt_to: Transport::Destination.new(rcpt_to, get_servers: false).address,
            message: preparer.to_s
          )

          # Store queue ID for logging or tracking
          queue_ids << message.queue[1]
        end

        # Send final response indicating message queued successfully
        message = SMTP::Response.new(
          status: :positive_completed,
          category: :mail_system,
          message: Config.active["queue"]["return_queue_ids"] ? "2.0.0 Message queued as #{queue_ids.join(?,)}" : "2.0.0 Message que
        )
        context.send_response(message)

        # Reset session state after processing message
        context.reset

        # Log queued message IDs
        Logger.log "Queued message as #{queue_ids.join(?,)}", origin: context.logger_origin, verbosity: 5
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_command_handlers/mailfrom.rb

```ruby
# Define the SMTPServer module containing nested modules for handlers
module SMTPServer
  module Handlers
    module SMTPCommandHandlers
      # This method processes the MAIL FROM command in SMTP protocol
      # Inputs:
      # - context: an object representing the current SMTP session state
      # - args: an array of arguments provided with the MAIL FROM command
      def self.mailfrom(context, args)
        # Check if the mailfrom attribute is already set in the context
        unless context.mailfrom
          # Log a warning if MAIL FROM command is received unexpectedly
          Logger.log "Received unexpected MAIL FROM: command", origin: context.logger_origin, verbosity: 5, type: :warn

          # Create a negative SMTP response indicating HELO/EHLO is required
          message = SMTP::Response.new(
            status: :negative_permanent,   # Permanent failure status
            category: :syntax,              # Indicates a syntax error
            detail: 3,                      # Specific detail code
            message: "5.5.1 Error: HELO/EHLO required"  # Error message
          )
          # Send the error response to the client
          context.send_response(message)
          return  # Exit the method after handling error
        end

        # Check if MAIL FROM command is being issued again without proper sequence
        if context.mailfrom != :ready
          # Create a negative response indicating repeated MAIL FROM command
          message = SMTP::Response.new(
            status: :negative_permanent,
            category: :syntax,
            detail: 3,
            message: "5.5.1 Error: repeated MAIL FROM: command"
          )
          # Send the error response
          context.send_response(message)
          return  # Exit the method
        end

        # Create a positive response indicating the command was successful
        message = SMTP::Response.new(
          status: :positive_completed,  # Success status
          category: :mail_system,
          message: "2.1.0 Ok"
        )
        # Send the success response
        context.send_response(message)

        # Set the mailfrom attribute in the context to the first argument
        context.mailfrom = args[0]
        # Initialize the recipient list as empty
        context.rcptto = []

        # Log the sender address for debugging or auditing
        Logger.log "Sender: `#{args[0]}`", origin: context.logger_origin, verbosity: 5
      end
    end
  end
end
```

lib/mlsmtp/handlers/smtp_server_handler.rb

```
# Define the SMTPServer module which groups related classes and modules
module SMTPServer
  # Define the Handlers module to contain various handler classes
  module Handlers
    # Define the SMTPServerHandler class to handle SMTP client interactions
    class SMTPServerHandler
      # Initialize the handler with a given context object
      # Input: context - an object providing connection and state information
      def initialize(context)
        @context = context  # Store the context for use in other methods
      end

      # Handle the SMTP client session
      # Input: context - the connection context object
      # Returns true if session ends positively, false if negatively
      def handle_client(context)
        # Read the initial server greeting message after connection
        server_banner = context.read_response

        # Log the server banner message with verbosity level 4
        Logger.log "Server ready: #{server_banner.message}", origin: context.logger_origin, verbosity: 4

        # Enter an indefinite loop to process SMTP commands
        while true
          # Determine which SMTP command to handle next
          ready_for = context.ready_for
          case ready_for
          when :ehlo
            # Handle EHLO command
            SMTPServerCommandHandlers.ehlo(context)
          when :helo
            # Handle HELO command
            SMTPServerCommandHandlers.helo(context)
          when :re_ehlo
            # Handle re-issue of EHLO without attempting STARTTLS
            SMTPServerCommandHandlers.ehlo(context, attempt_starttls: false)
          when :starttls
            # Handle STARTTLS command to initiate TLS encryption
            SMTPServerCommandHandlers.starttls(context)
          when :mailfrom
            # Handle MAIL FROM command to specify sender
            SMTPServerCommandHandlers.mailfrom(context)
          when :rcptto
            # Handle RCPT TO command to specify recipient
            SMTPServerCommandHandlers.rcptto(context)
          when :data
            # Handle DATA command to send email content
            SMTPServerCommandHandlers.data(context)
          when :quit_positive
            # Handle QUIT command with positive response and end session
            SMTPServerCommandHandlers.quit(context)
            return true  # Indicate successful session termination
          when :quit_negative
            # Handle QUIT command with negative response and end session
            SMTPServerCommandHandlers.quit(context, negative: true)
            return false  # Indicate session ended negatively
          end
        end
      end
    end
  end
end
```

lib/mlsmtp/transport/rules.rb

```
module SMTPServer
  module Transport
    # The Rules class manages routing rules for email address handling
    class Rules
      @@active_rules = nil # Class variable to hold the currently active rules instance

      # Initialize with the rules file path
      def initialize(rules_file)
        @file = rules_file # Store the rules file path
        @rules = JSON.parse(File.read(@file)) # Load and parse rules from the JSON file
      end

      # Determine the destination for a given email address
      # Input: address string
      # Output: array containing substituted user and server info or original address with domain
      def determine_destination(address)
        user, domain = address.split(?@, 2) # Split address into user and domain parts
        @rules.each do |rule|
          regex, destination = rule # Each rule is a pair: regex pattern and destination info

          if address.match?(Regexp.new(regex))
            # If address matches the regex, process the destination substitution
            dest_user = substitute_specials(
              destination[0], # Destination user pattern
              user: user,
              domain: domain,
              address: address
            )

            if destination[1]
              # If server address is specified, wrap it in a hash
              dest_server = {server_addrs: destination[1]}
            else
              # Otherwise, no server info
              dest_server = nil
            end

            return [ dest_user, dest_server ] # Return the computed destination info
          end
        end

        # If no rule matches, return original address and its domain info
        return [ address, {mail_domain: domain} ]
      end

      # Set this Rules instance as the active rules
      def set_active
        @@active_rules = self
      end

      # Class method to retrieve the current active rules
      def self.active
        @@active_rules
      end

      # Class method to clear the active rules
      def self.clear_active
        @@active_rules = {}
      end

      # Instantiate a Rules object with the rules file from configuration and set it active
      new(Config.active["transport"]["rules_file"]).set_active

      attr_accessor :file, :rules # Accessors for file path and rules data
```

```
      private

      # Substitute special patterns in the string with actual values
      # Inputs: string with placeholders, user, domain, address
      # Output: string with placeholders replaced
      def substitute_specials(string, user:, domain:, address:)
        string
          .gsub("%u", user) # Replace user placeholder
          .gsub("%d", domain) # Replace domain placeholder
          .gsub("%a", address) # Replace address placeholder
      end
    end
  end
end
```

lib/mlsmtp/transport/destination.rb

```
# Module SMTPServer contains the nested module Transport which defines the class Destination
module SMTPServer
  module Transport
    # The Destination class models an email destination including address parsing and server resolution
    class Destination
      # Initialize the Destination object with an email address and optional server fetching
      # address: the email address to process
      # get_servers: boolean indicating whether to retrieve destination servers
      def initialize(address, get_servers: true)
        @address = fix_address(address) # Clean and standardize the input address
        @addr_user, @addr_domain = @address.split(?@) # Split address into user and domain parts
        @destination = Rules.active.determine_destination(@address) # Determine destination info via rules
        @destination_user = @destination[0] # Extract the user part from destination info

        # If destination includes server addresses, fetch them if get_servers is true
        if @destination[1]
          @destination_servers = get_servers ? get_destination_servers(@destination[1]) : []
        else
          @local = true # Mark as local if no destination servers are provided
        end

        # If the destination is local, sanitize the user part by removing dots and plus signs
        if @local
          @destination_user = @destination_user.gsub(?., "").split(?+)[0]
        end
      end

      # Returns the email address enclosed in angle brackets
      def formatted_address
        "<#{@address}>"
      end

      # Read-only accessors for various instance variables
      attr_reader :address, :destination, :destination_user, :destination_servers, :local, :addr_user, :addr_domain

      private

      # Retrieve destination server addresses based on destination info
      # destination: hash containing server_addrs array or mail_domain string
      # Returns an array of server addresses
      def get_destination_servers(destination)
        dest_servers = [] # Initialize empty server list

        # If server addresses are explicitly provided, use them
        if !(destination[:server_addrs].to_a.empty?)
          dest_servers = destination[:server_addrs]
        # Else if a mail domain is specified, fetch MX records for it
        elsif destination[:mail_domain]
```

```
        elsif destination[:mail_domain]
          dest_servers = get_mx_records(destination[:mail_domain])
        end

        return dest_servers # Return the list of server addresses
      end

      # Fetch MX records for a given domain
      # domain: string representing the mail domain
      # Returns an array of server hostnames sorted by preference
      def get_mx_records(domain)
        # Handle literal IP addresses enclosed in brackets
        if domain.match?(/\[.*\]/)
          return [domain[1..-2]] # Remove brackets and return as single-element array
        end

        begin
          # Use DNS resolver to get MX records for the domain
          mx_records = Resolv::DNS.open do |dns|
            dns.getresources(domain, Resolv::DNS::Resource::IN::MX)
          end
        rescue
          mx_records = [] # Return empty list if DNS lookup fails
        end

        # Map MX records to hostname strings, sorted by preference
        return mx_records.map{|x| [x.preference, x.exchange.to_s]}.sort_by(&:first).map(&:last)
      end

      # Standardize and clean the email address
      # address: the raw email address string
      # Returns a cleaned email address string
      def fix_address(address)
        # If address is enclosed in angle brackets, extract inner part
        if address.match?(/^\<.*\>$/)
          inner_address = address[1..-2]
        # Else if address contains a less-than sign, extract the part after it
        elsif address.split(?<)[-1].match?(/^.*\>$/)
          inner_address = address.split(?<)[-1][0..-2]
        else
          inner_address = address # Otherwise, use the address as-is
        end

        # If address lacks an at symbol, assign a default domain
        if address.split(?@).length == 1
          return "#{inner_address}@0.0.0.0" # Append default domain
        else
          return inner_address # Return the cleaned address
        end
      end
    end
  end
end
```

lib/mlsmtp/transport/remote_delivery_agent.rb

```
module SMTPServer
  module Transport
    # This class handles remote email delivery to SMTP servers
    class RemoteDeliveryAgent
      # Class variables for configuration: list of ports to attempt and socket timeout duration
      @@attempt_ports = Config.active["external_transport"]["attempt_ports"]
      @@timeout = Config.active["external_transport"]["socket_timeout"]

      # Initializes the agent with destination info, sender, message, and origin for logging
```

```ruby
# Inputs:
# - destination: object containing destination server info and user
# - sender: sender email address
# - message: email message content
# - origin: context info for logging
def initialize(destination:, sender:,  message:, origin:)
  @message = message
  @destination = destination
  @servers = @destination.destination_servers  # List of destination servers
  @recipient_addr = @destination.destination_user  # Recipient email address
  @sender_addr = sender  # Sender email address
  @origin = origin  # Origin for logging purposes
end

# Attempts to deliver the message to one of the destination servers on specified ports
# Returns true if delivery succeeds, raises error if all attempts fail
def attempt_delivery
  # Loop through each port to try connection
  @@attempt_ports.each do |port|
    # Loop through each server for the current port
    @servers.each do |server|
      # If delivery to server succeeds, return true
      return true if deliver_to_server(server, port)
    end
  end
  # Raise error if delivery failed on all server and port combinations
  raise Errors::ServerRejectionError
end

# Accessors for message, destination, servers, and recipient address
attr_accessor :message, :destination, :servers, :recipient_addr

private

# Handles connection and communication with a specific server at a specific port
# Inputs:
# - server: server hostname or IP address
# - port: port number to connect to
# Returns true if SMTP client handling succeeds, false otherwise
def deliver_to_server(server, port)
  # Log attempt to connect to server
  Logger.log "Trying server #{server}:#{port}", origin: @origin, verbosity: 3
  tcp_server = nil

  begin
    # Attempt to open TCP connection within the specified timeout
    Timeout.timeout(@@timeout) do
      tcp_server = TCPSocket.new(server, port)
    end
  rescue Errno::ECONNREFUSED
    # Log connection refused error and return false
    Logger.log "Connection to #{server}:#{port} refused", origin: @origin, verbosity: 3, type: :warn
    return false
  rescue Timeout::Error
    # Log timeout error and return false
    Logger.log "#{server}:#{port} timed out in #{@@timeout} seconds", origin: @origin, verbosity: 3, type: :warn
    return false
  end

  # Log creation of SMTP context for communication
  Logger.log "Creating context for server", origin: @origin, verbosity: 3
  # Initialize SMTP server context with the TCP socket
  context = SMTP::SMTPServerContext.new(tcp_server)
  # Set recipient, sender, and message data in context
  context.recipient_addr = @recipient_addr
  context.sender_addr = @sender_addr
```

```
        context.sender_addr = @sender_addr
        context.data = @message

        # Log handling of server communication
        Logger.log "Handling server", origin: @origin, verbosity: 3
        # Instantiate SMTP server handler with the context
        handler = Handlers::SMTPServerHandler.new(context)
        # Handle SMTP client interaction and return success or failure
        return handler.handle_client(context)
      end
    end
  end
end
```

lib/mlsmtp/transport/local_delivery_agent.rb

```
# Define the SMTPServer module to encapsulate related classes and modules
module SMTPServer
  # Define the Transport module within SMTPServer to group transport-related classes
  module Transport
    # Define the LocalDeliveryAgent class responsible for delivering messages locally
    class LocalDeliveryAgent
      # Initialize with user and message objects, setting instance variables
      def initialize(user:, message:)
        @user = user # Store the user object for delivery
        @message = message # Store the message object to be delivered
      end

      # Method to attempt delivery of the message to the user
      def attempt_delivery
        Storage::Active.add_message(@user, @message) # Add message to storage for the user
      end

      # Provide read access to user and message attributes
      attr_reader :user, :message
    end
  end
end
```

lib/mlsmtp/transport/authorization.rb

```
module SMTPServer
  module Transport
    class Authorization
      # Class variable to hold the current active authorization instance
      @@active_authorization = nil

      # Initialize method reads authorization data from a file and parses it as JSON
      # Inputs:
      #   authorization_file - path to the authorization JSON file
      def initialize(authorization_file)
        @file = authorization_file
        @authorization = JSON.parse(File.read(@file))
      end

      # Method to check if the sender and recipients are authorized based on rules
      # Inputs:
      #   context - object containing mailfrom, ip_addr, authenticated_as, and rcptto
      # Output:
      #   true if authorized, false otherwise
      def authorized?(context)
        # Extract sender email address from context and parse user and domain
        sender_address = Destination.new(context.mailfrom, get_servers: false).address
        sender_user, sender_domain = sender_address.split(?@)
```

```
  # Convert sender IP address to IPAddr object for comparison
  sender_ip = IPAddr.new(context.ip_addr)
  # Get the authentication status of the sender
  authenticated_as = context.authenticated_as

  # Iterate over each recipient in the context
  context.rcptto.each do |recipient|
    # Create destination object for recipient and parse address components
    destination = Destination.new(recipient, get_servers: false)
    destination_address = destination.address
    destination_user, destination_domain = destination_address.split(?@)

    # Iterate over each authorization rule
    @authorization.each do |auth_rule|
      # Extract rule type and authorization exemption flag
      rule_type = auth_rule["rule"].to_s.downcase
      auth_exempt = auth_rule["auth_exempt"] || false

      # Validate rule type; raise error if invalid
      raise Errors::BadAuthRuleError, rule_type unless [ "allow", "deny" ].include?(rule_type)

      # Extract matching and determining conditions from rule
      match_conditions = auth_rule["match_by"]
      determine_conditions = auth_rule["determine_by"]

      # Check if current rule matches the sender and recipient based on match conditions
      next unless matches?(
        match_conditions,
        destination_user: destination_user,
        destination_domain: destination_domain,
        destination_address: destination_address,
        sender_user: sender_user,
        sender_domain: sender_domain,
        sender_address: sender_address,
        sender_ip: sender_ip,
        authenticated_as: authenticated_as
      )

      # Determine the result of the rule based on determine conditions
      result = matches?(
        determine_conditions,
        destination_user: destination_user,
        destination_domain: destination_domain,
        destination_address: destination_address,
        sender_user: sender_user,
        sender_domain: sender_domain,
        sender_address: sender_address,
        sender_ip: sender_ip,
        authenticated_as: authenticated_as
      )

      # Invert result if rule is a deny rule
      result = !result if rule_type == "deny"

      # Set context's authorization exemption flag
      context.authorization_exempt = auth_exempt

      # Return the final authorization decision
      return result
    end
  end

  # Default to false if no rules match
  return false
end
```

```
end

# Class method to set the current authorization instance as active
def set_active
  @@active_authorization = self
end

# Class method to retrieve the current active authorization instance
def self.active
  @@active_authorization
end

# Class method to clear the active authorization instance
def self.clear_active
  @@active_authorization = nil
end

# Attribute accessors for the authorization file and data
attr_accessor :file, :authorization

private

# Helper method to substitute special placeholders with actual values
# Inputs:
#   string - string containing placeholders
#   user - user part of email address
#   domain - domain part of email address
#   address - full email address
# Output:
#   string with placeholders replaced by actual values
def substitute_specials(string, user:, domain:, address:)
  string
    .gsub("%u", user)
    .gsub("%d", domain)
    .gsub("%a", address)
end

# Method to check if an authorization rule matches the current context
# Inputs:
#   auth_rule - hash representing a single authorization rule
#   destination_user - recipient username
#   destination_domain - recipient domain
#   destination_address - recipient email address
#   sender_user - sender username
#   sender_domain - sender domain
#   sender_address - sender email address
#   sender_ip - sender IP address as IPAddr object
#   authenticated_as - authentication status or user
# Output:
#   true if the rule matches, false otherwise
def matches?(auth_rule, destination_user:, destination_domain:, destination_address:, sender_user:, sender_domain:, sender_add
  # Extract allowed sender IPs from rule
  allowed_sender_ips = auth_rule["from_ip"]

  # Compile regular expressions for from_email and to_email if specified
  from_email_re = auth_rule["from_email"] ?  Regexp.new(auth_rule["from_email"]) : nil
  to_email_re = auth_rule["to_email"] ?  Regexp.new(auth_rule["to_email"]) : nil

  # Check if sender address matches from_email pattern
  sender_matches = from_email_re ? sender_address.match?(from_email_re) : true
  # Check if recipient address matches to_email pattern
  recipient_matches = to_email_re ? destination_address.match?(to_email_re) : true

  # Determine authentication matching based on auth field
  if auth_rule["auth"].nil?
    auth_matches = true
```

```
        elsif auth_rule["auth"] == false
          auth_matches = authenticated_as == false
        else
          # Substitute placeholders in auth string with actual values
          required_auth = substitute_specials(
            auth_rule["auth"],
            user: sender_user,
            domain: sender_domain,
            address: sender_address
          )
          # Check if authenticated user matches required auth
          auth_matches = required_auth == authenticated_as
        end

        # Check if sender IP is within allowed IPs if specified
        if allowed_sender_ips
          ip_matches = allowed_sender_ips.any? do |ip|
            IPAddr.new(ip).include?(sender_ip)
          end
        else
          ip_matches = true
        end

        # Final match is true only if all individual matches are true
        return sender_matches && recipient_matches && ip_matches && auth_matches
      end

      # Instantiate the class with the active configuration and set as active
      new(Config.active["transport"]["authorization_file"]).set_active
    end
  end
end
```

lib/mlsmtp/transport/authorization_handler.rb

```
# Define the SMTPServer module to encapsulate server-related classes and modules
module SMTPServer
  # Define the Transport module to group transport-related classes
  module Transport
    # Define the AuthorizationHandler class to manage authorization process
    class AuthorizationHandler
      # Initialize the handler with a given context object
      # context - an object containing the current session or connection information
      def initialize(context)
        @context = context
      end

      # Handle the authorization process based on the current context
      def handle_authorization
        # Check if the authorization is active and the current context is authorized
        authorized = Authorization.active.authorized?(@context)

        if authorized
          # Log successful authorization with specified origin and verbosity level
          Logger.log "Authorization passed", origin: @context.logger_origin, verbosity: 5

          # Set the context data to indicate readiness
          @context.data = :ready

          # Create a positive SMTP response indicating successful authorization
          response = SMTP::Response.new(
            status: :positive_completed, # Status code for success
            category: :mail_system,     # Category of the response
            message: "2.1.5 Ok"         # Human-readable message
          )

          # Send the positive response back to the client
          @context.send_response(response)
        else
          # Log failed authorization attempt with warning level
          Logger.log "Authorization failed", origin: @context.logger_origin, verbosity: 5, type: :warn

          # Create a negative SMTP response indicating authorization failure
          response = SMTP::Response.new(
            status: :negative_permanent, # Permanent failure status code
            category: :mail_system,     # Category of the response
            detail: 3,                  # Additional detail code
            message: "5.7.1 Authorization Error" # Human-readable error message
          )

          # Send the negative response back to the client
          @context.send_response(response)
        end
      end
    end
  end
end
```

lib/mlsmtp/smtp/smtp_client_context.rb

```
# This module defines an SMTP client context class for handling SMTP protocol communication over a TCP connection.
module SMTPServer
  module SMTP
    # The SMTPClientContext class manages the state and communication with an SMTP client.
    class SMTPClientContext
      # Initialize the SMTP client context with a TCP client connection.
      # Inputs:
      # - client: the TCP socket connected to the SMTP client.
      def initialize(client)
```

```
    @tcp_client = client # Store the TCP client socket.
    @client = @tcp_client # Alias for easier reference.
    @banner_sent = false # Flag to indicate whether the SMTP banner has been sent.
    @closed = false # Flag to indicate whether the connection is closed.
    # Set mailfrom status based on configuration; false if require_helo is true, else ready.
    @mailfrom = (Config.active["require_helo"] ? false : :ready)
    @ip_addr = @client.peeraddr[3] # Extract client's IP address.
    @logger_origin = "Client Handler: #{@ip_addr}" # String for logging origin.
    # Determine support for 8-bit data based on configuration.
    @use_8bit = Config.active["support_8_bit"] && Config.active["esmtp_enable"]
    @esmtp = false # Flag to indicate if ESMTP is active.
    @starttls_support = false # Flag for STARTTLS support.
    @using_starttls = false # Flag to indicate if STARTTLS is currently in use.
    @starttls_certificate = nil # Placeholder for STARTTLS certificate data.

    initialize_statuses # Set initial statuses for the session.
  end

  # Send the SMTP banner message to the client.
  # Returns false if connection is closed, otherwise sends banner and marks it as sent.
  def send_banner
    return false if @closed # Do not send if connection is closed.
    banner = Banner.new.to_response # Generate the banner response.
    send_response(banner) # Send the banner to the client.
    @banner_sent = true # Mark banner as sent.
  end

  # Send a response message to the client.
  # Inputs:
  # - response: a string, array of strings, or Response object to send.
  # Returns false if connection is closed.
  def send_response(response)
    return false if @closed # Do not send if connection is closed.
    # Convert response to array of lines if it's a string.
    response = response.split("\r\n") if response.is_a?(String)
    # Convert Response object to array if needed.
    response = response.to_a if response.is_a?(Response)
    # Send each line to the client socket.
    response.each do |rline|
      @client.print "#{rline}\r\n" # Append CRLF as per SMTP protocol.
    end
  end

  # Reset the session statuses to initial state.
  def reset
    initialize_statuses # Reinitialize statuses.
  end

  # Close the client connection and mark as closed.
  def close
    @closed = true # Mark as closed.
    @client.close # Close the TCP socket.
  end

  # Read data lines from the client until read_until string is encountered or connection closes.
  # Inputs:
  # - read_until: optional string to stop reading at.
  # Returns:
  # - Array of lines read, or false if connection is closed.
  def read(read_until: nil)
    return false if @closed # Do not read if connection is closed.
    lines = [] # Initialize array to store lines.

    while true
      line = @client.gets # Read a line from the socket.
```

```
          # If 8-bit support is not enabled, encode line to US-ASCII replacing invalid characters.
          unless @use_8bit
            line = line.encode('US-ASCII', invalid: :replace, undef: :replace, replace: '?')
          end

          if line
            line = line.chomp # Remove trailing newline characters.
          else
            @closed = true # Connection closed unexpectedly.
            return [""] # Return array with empty string.
          end

          break if line == read_until # Stop reading if the read_until string matches.
          lines << line # Append the line to the array.
          break unless read_until # Exit if no read_until specified.
        end

        return lines # Return the array of lines read.
      end

      # Define accessor methods for various session variables.
      attr_accessor :mailfrom, :rcptto, :data, :done, :closed, :banner_sent, :heloname, :ip_addr, :logger_origin, :esmtp, :starttls_

      private

      # Initialize or reset session statuses to default values.
      def initialize_statuses
        @done = false # Flag indicating if session is complete.
        # Set mailfrom status to ready if already set, else false.
        @mailfrom = @mailfrom ? :ready : false
        @heloname = nil # Placeholder for HELO/EHLO name.
        @rcptto = false # Recipient acceptance status.
        @data = false # Flag indicating if data command has been received.
        @authenticated_as = nil # Store authentication info if applicable.
        @additional_authorization_data = {} # Placeholder for extra auth data.
        @authorization_exempt = false # Flag for authorization exemption.
      end
    end
  end
end
```

lib/mlsmtp/smtp/smtp_server_context.rb

```ruby
module SMTPServer
  module SMTP
    # This class manages the context of an SMTP server connection for a client.
    class SMTPServerContext
      # Initializes the context with the server socket object.
      # Sets default states and extracts client IP address.
      # Inputs:
      #   server - the TCP socket object representing the client connection
      def initialize(server)
        @tcp_server = server # Store the TCP server socket
        @server = @tcp_server # Alias for easier reference
        @ready_for = :ehlo # Initial state expecting EHLO command
        @recipient_addr = nil # Placeholder for recipient email address
        @sender_addr = nil # Placeholder for sender email address
        @data = nil # Placeholder for email data

        @ip_addr = @server.peeraddr[3] # Extract client's IP address
        @logger_origin = "Server Handler: #{@ip_addr}" # String for logging purposes
      end

      # Sends response data to the client.
      # Converts Command objects to strings and ensures response is an array.
      # Inputs:
      #   response - string, Command object, or array of responses to send
      def send_data(response)
        response = response.to_s if response.is_a?(Command) # Convert Command to string
        response = [ response ] unless response.is_a?(Array) # Wrap single response in array

        # Send each line to the client with CRLF line ending
        response.each do |line|
          @server.print "#{line}\r\n"
        end
      end

      # Reads the response from the client until a complete response is received.
      # Returns the parsed Response object.
      def read_response
        response_lines = [] # Collect response lines from client

        # Loop to read multiple lines if necessary
        while true
          response_lines << @server.gets.chomp # Read a line and remove trailing newline
          response = Response.parse(response_lines) # Parse the response lines

          # Exit loop if response is complete
          return response unless response == :incomplete
        end
      end

      # Closes the client connection.
      def close
        @server.close # Close the TCP socket
      end

      # Read-only attributes for IP address and logger origin
      attr_reader :ip_addr, :logger_origin
      # Read-write attributes for server state and connection details
      attr_accessor :ready_for, :recipient_addr, :sender_addr, :data, :tcp_server, :server
    end
  end
end
```

lib/mlsmtp/smtp/commands.rb

```
 # This module defines classes and methods for handling SMTP commands in a server context.
# It validates, parses, and constructs SMTP command strings based on predefined command templates.

module SMTPServer
  module SMTP
    class Command
      # List of valid SMTP commands with optional expected argument types or prefixes
      VALID_SMTP_COMMANDS = [
        [ "HELO", String ],             # HELO command with a domain argument
        [ "MAIL", "FROM:", String ],    # MAIL command with FROM prefix and address
        [ "RCPT", "TO:", String ],      # RCPT command with TO prefix and recipient address
        [ "DATA" ],                     # DATA command without arguments
        [ "QUIT" ],                     # QUIT command
        [ "RSET" ],                     # RSET command
        [ "VRFY", String ],             # VRFY command with a string argument
        [ "EXPN", String ],             # EXPN command with a string argument
        [ "NOOP" ],                     # NOOP command
      ]

      # List of valid ESMTP commands with optional arguments
      VALID_ESMTP_COMMANDS = [
        [ "EHLO", String ],             # EHLO command with domain argument
        [ "STARTTLS" ],                 # STARTTLS command
        [ "AUTH", String ],             # AUTH command with mechanism argument
      ]

      # Cache for all valid commands depending on configuration
      @@all_valid_commands = nil

      # Initialize a Command object with command string and optional values
      # Inputs:
      # - command: String or Array representing the command
      # - values: optional array of arguments or parameters
      def initialize(command, values = [])
        @command = command
        @values = values

        # Ensure command is an array of parts
        @command = @command.split(" ") unless @command.is_a?(Array)
        # Ensure values is an array
        @values = [ @values ] unless @values.is_a?(Array)

        # Convert command parts to uppercase for standardization
        @command.map!(&:upcase)

        # Parse the command string to validate and structure it
        Command.parse_command(to_s)
      end

      # Convert command object back to string form
      def to_s
        "#{@command.join(" ")}#{" " unless @values.empty?}#{@values.join(" ")}"
      end

      # Parse a full command string into a Command object
      # Input:
      # - full_command: String representing the entire command line
      # Output:
      # - new Command object with parsed command and values
      def self.parse(full_command)
        parsed_command, command_template = parse_command(full_command)

        command, values = dissect_array(parsed_command, command_template)

        new(command, values)
      end
```

```
end

# Retrieve all valid commands considering current configuration
# Output:
# - Array of command templates allowed
def self.all_valid_commands
  unless @@all_valid_commands
    # Start with standard SMTP commands
    @@all_valid_commands = VALID_SMTP_COMMANDS
    # Add ESMTP commands if enabled
    @@all_valid_commands += VALID_ESMTP_COMMANDS if Config.active["esmtp_enable"]

    # Filter out commands that are disabled in configuration
    @@all_valid_commands = @@all_valid_commands.filter do |command|
      !Config.active["disable_commands"].include?(command[0])
    end
  end

  @@all_valid_commands
end

attr_accessor :command, :values

private

# Split command string into array considering special argument syntax
# Inputs:
# - command: String of command parts
# - command_template: Array template for command validation
# Output:
# - Array of command parts, possibly with adjusted argument parts
def self.split_to_array(command, command_template)
  command = command.split(" ")

  # Handle cases where argument is attached with colon, e.g., MAIL FROM:
  if command_template.length >= 2 && command.length >= 2 && command_template[1].is_a?(String) && command_template[1][-1] == ?:
    # Split the second part at colon
    command_end, argument = command.delete_at(1).split(?:)
    # Reinsert with colon attached
    command.insert(1, "#{command_end}:")
    command.insert(2, argument)
  end

  return command
end

# Dissect command array into command parts and arguments based on template
# Inputs:
# - command_array: Array of command parts
# - template_array: Array template with expected argument types
# Output:
# - Array with command parts and arguments separated
def self.dissect_array(command_array, template_array)
  args_start = template_array.index(String)
  # If no arguments expected, return entire command as single element
  return [ command_array ] unless args_start

  command_strings_end = args_start - 1

  [
    # Command parts before arguments
    command_array[0..command_strings_end],
    # Arguments parts
    command_array[args_start..-1]
  ]
end
```

```
      # Parse the command string against valid command templates
      # Inputs:
      # - command: String command to parse
      # - No explicit output; raises error or returns parsed command
      def self.parse_command(command)
        # Iterate through all valid command templates
        Command.all_valid_commands.each do |command_template|
          case command_matches_array?(command, command_template)
          when true
            # If match, split command into parts and return with template
            return [
              split_to_array(command, command_template),
              command_template
            ]
          when :incomplete
            # Raise error if command is incomplete
            raise Errors::IncompleteCommandError, [command, command_template]
          when false
            # Continue to next template if no match
            next
          end
        end

        # Raise error if no valid command matches
        raise Errors::InvalidCommandError, command
      end

      # Check if a command string matches a command template
      # Inputs:
      # - command: String command to check
      # - array: Template array to match against
      # Output:
      # - true if matches, false if invalid, :incomplete if incomplete
      def self.command_matches_array?(command, array)
        command = split_to_array(command, array)

        # Iterate through each element in the template
        array.each_with_index do |element, index|
          value = command[index]

          if element.class == String
            # For string elements, compare ignoring case and spaces
            return false unless value.to_s.upcase.gsub(" ", "") == element
          else
            # For argument placeholders, check if value matches expected class
            if element != value.class
              return :incomplete
            end
          end
        end

        return true
      end
    end
  end
end
```

lib/mlsmtp/smtp/banner.rb

```
# Define the SMTPServer module containing the SMTP submodule
module SMTPServer
  module SMTP
    # Define the Banner class which constructs the SMTP server banner message
    class Banner
      # Initialize the Banner object with configuration values
      # No inputs; sets instance variables based on configuration
      def initialize
        # Determine whether to include ESMTP status based on configuration
        @include_esmtp = Config.active["banner"]["include_esmtp_status"] && Config.active["esmtp_enable"]
        # Additional text to append to the banner, from configuration
        @additional_text = Config.active["banner"]["additional_text"]
        # Server name to display in the banner
        @servername = Config.active["banner"]["banner_server_name"]
        # Mail name used as the prefix in the banner
        @mailname = Config.active["mailname"]
        # Optional message override to replace the default banner text
        @messageoverride = Config.active["banner"]["message_override"]
      end

      # Convert the Banner object to its string representation
      # No inputs; returns the formatted banner string
      def to_s
        # Prepare ESMTP prefix if included
        esmtp_text = @include_esmtp ? "ESMTP " : ""
        # Store server name
        servername_text = @servername
        # Store additional text
        additional_text = @additional_text

        # Use message override if present; otherwise, build banner text
        if @messageoverride
          bannertext = @messageoverride
        else
          # Concatenate ESMTP prefix, server name, and additional text
          bannertext = "#{esmtp_text}#{servername_text}#{additional_text}"
        end

        # Add leading space if banner text is not empty
        bannertext = " #{bannertext}" unless bannertext.empty?

        # Concatenate mailname and banner text, then strip whitespace
        return "#{@mailname}#{bannertext}".strip
      end

      # Generate a response object containing the banner message
      # No inputs; returns a Response object with status, category, and message
      def to_response
        Response.new(
          status: :positive_completed,  # Indicate successful connection
          category: :connections,       # Category of response
          message: to_s                 # The banner message string
        )
      end
    end
  end
end
```

lib/mlsmtp/smtp/responses.rb

```
# This module defines an SMTP server response class to handle SMTP response codes and messages
module SMTPServer
  module SMTP
    class Response
```

```
# Regular expression to validate SMTP response codes which are three digits starting with 1-5
VALID_CODE_REGEX = /^[1-5][0-5][0-9]$/

# Hash mapping symbolic status names to their numeric codes
@@statuses = {
  information: 1,
  positive_completed: 2,
  positive_intermediate: 3,
  negative_temporary: 4,
  negative_permanent: 5
}

# Hash mapping symbolic category names to their numeric codes
@@categories = {
  syntax: 0,
  information: 1,
  connections: 2,
  authentication: 3,
  unspecified: 4,
  mail_system: 5
}

# Initialize a Response object with either status, category, detail or a full code and optional message
# Inputs:
# - status: symbol or integer representing the response status
# - category: symbol or integer representing the category
# - detail: integer representing the detail code
# - code: string or integer representing the full response code
# - message: optional message string or array of strings
def initialize(status: nil, category: nil, detail: 0, code: nil, message: nil)
  if status && category && detail
    # Convert symbolic status and category to their numeric codes if needed
    status = @@statuses[status] if status.is_a?(Symbol)
    category = @@categories[category] if category.is_a?(Symbol)

    # Validate status is between 1 and 5
    raise Errors::BadCodeError, [:status, status] unless (1..5).include?(status)
    # Validate category is between 0 and 5
    raise Errors::BadCodeError, [:category, category] unless (0..5).include?(category)
    # Validate detail is between 0 and 9
    raise Errors::BadCodeError, [:detail, detail] unless (0..9).include?(detail)

    @status, @category, @detail = [status, category, detail]
  elsif code
    # Validate the code matches the SMTP response code pattern
    raise Errors::BadCodeError, [:fullcode, code] unless code.to_s.match?(VALID_CODE_REGEX)

    # Split the code into individual digits for status, category, and detail
    @status, @category, @detail = code.to_s.split("").map(&:to_i)
  else
    # Collect missing elements for error reporting
    missing_codes = []
    missing_codes << :status unless status
    missing_codes << :category unless category
    missing_codes << :detail unless detail

    # Raise error if required elements are missing
    raise Errors::BadCodeError, [:missingelements, missing_codes]
  end

  # Assign the message, which can be a string or array of strings
  @message = message
end

# Generate the full response code as a string combining status, category, and detail
def code
```

```
def code
  "#{@status}#{@category}#{@detail}"
end

# Convert the response to a string suitable for SMTP response output
def to_s
  to_a.join("\r\n")
end

# Convert the response to an array of response lines, handling multiline messages
def to_a
  if @message.is_a?(Array)
    response = []

    # Iterate through each message line with its index
    @message.each_with_index do |line, index|
      if index != 0 && index == message.length - 1
        # For the last line in a multiline message, prepend the code and a space
        response << "#{code} #{line}"
      else
        # For other lines, prepend the code and a hyphen
        response << "#{code}-#{line}"
      end
    end
  else
    # For single-line messages, prepend the code and optional message
    message = @message ? " #{@message}" : nil
    response = [ "#{code}#{message}" ]
  end

  return response
end

# Class method to parse a response string or array into a Response object
# Input:
# - response: string or array of response lines
# Output:
# - Response object or symbol :incomplete if multiline message is incomplete
def self.parse(response)
  # Split string input into lines if necessary
  response = response.split(/\r?\n/) if response.is_a?(String)

  # Check if the response is an incomplete multiline message
  if is_multiline_incomplete?(response[0]) && is_multiline_incomplete?(response[-1])
    return :incomplete
  end

  # Extract the code from the first line, splitting on hyphen or space
  code = response[0].split(/[\-\s]/, 2)[0]
  # Extract message lines by removing the code prefix
  message = response.map{ |rline| rline.split(/[\-\s]/, 2)[-1] }

  # If only one message line, simplify to a single string
  message = message[0] if message.length == 1

  # Create and return a new Response object with parsed code and message
  new(
    code: code,
    message: message
  )
end

# Attribute accessors for response properties
attr_accessor :status, :category, :detail, :message

private
```

```
      # Helper method to determine if a multiline response is incomplete
      # Inputs:
      # - line: string representing the response line
      # Output:
      # - boolean indicating if the line indicates an incomplete message
      def self.is_multiline_incomplete?(line)
        # Checks if the fourth character of the line is a hyphen
        line[3] == ?-
      end
    end
  end
end
```

lib/mlsmtp/email/headers/received.rb

```
module SMTPServer  # Defines the main module for SMTP server related classes and modules
  module Email       # Defines a submodule for email related functionalities
    module Headers   # Defines a submodule for email header related classes
      class Received  # Defines a class to generate Received email header entries
        # Initializes the Received object with context information
        # context - an object providing heloname, ip_addr, and esmtp attributes
        def initialize(context)
          @received_helo = context.heloname  # Stores the HELO/EHLO hostname from the context
          @received_ip = context.ip_addr     # Stores the IP address from the context
          @esmtp = context.esmtp             # Stores whether ESMTP was used from the context
        end

        # Converts the Received object into a string representing a header line
        def to_s
          "from #{@received_helo} (#{@received_helo} [#{@received_ip}]) " +  # Formats the from part with hostname and IP
          "by #{Config.active["mailname"]} (#{Config.active["banner"]["banner_server_name"]}) " +  # Formats the by part with server
          "with #{@esmtp ? "ESMTP" : "SMTP"} " +  # Indicates whether ESMTP or SMTP was used
          "#{Time.now.strftime("%a, %d %b %Y %H:%M:%S %z")}"  # Appends the current timestamp in RFC format
        end
      end
    end
  end
end
```

lib/mlsmtp/email/error_email_generator.rb

```
# Define the SMTPServer module to encapsulate related classes and modules
module SMTPServer
  # Define the Email module to group email-related classes
  module Email
    # Define the ErrorEmailGenerator class responsible for generating and queuing error emails
    class ErrorEmailGenerator
      # Initialize the generator with error details and email context
      # Inputs:
      # - error: the exception object encountered during email processing
      # - origin: the source or context of the email
      # - mail_from: the sender's email address
      # - rcpt_to: the recipient's email address
      # - is_error_response: flag indicating if the email is an error response (0 or 1)
      def initialize(error, origin:, mail_from:, rcpt_to:, is_error_response:)
        @error = error
        @origin = origin
        @mail_from = mail_from
        @rcpt_to = rcpt_to
        @is_error_response = is_error_response
      end
```

```
      # Method to determine error type, log, and queue appropriate error emails
      def queue_email
        # Default error type to other_internal
        error = :other_internal
        # Map specific error classes to error string identifiers
        error = "bad_mailbox" if @error.class == SMTPServer::Errors::NonexistentMailboxError
        error = "delivery_timeout" if @error.class == Timeout::Error
        error = "delivery_failed" if @error.class == SMTPServer::Errors::ServerRejectionError

        # Log unexpected errors not mapped to specific error types
        if error == :other_internal
          Logger.log "Unexpected error when delivering email: #{@error}", origin: @origin, verbosity: 3, type: :warn
        else
          # Log the specific error encountered during email delivery
          Logger.log "Error delivering email: `#{@error}`", origin: @origin, verbosity: 3, type: :warn

          # Create a new error email object with original sender, recipient, and error type
          error_email = Email::ErrorEmail.new(
            original_from: @mail_from,
            original_to: @rcpt_to,
            email_name: error
          )

          # Prepare the email content for the error email
          err_email_text = error_email.prepared_email

          # Check if the email is not an error response and if the email content exists
          if @is_error_response == 0 && err_email_text
            # Create a queued message with contact email as sender and original sender as recipient
            queued_response = Queue::QueuedMessage.new(
              mail_from: Config.active["contact_email"],
              rcpt_to: @mail_from,
              message: err_email_text,
              error_response: true
            )

            # Retrieve the ID of the queued message
            id = queued_response.queue[1]

            # Log the queuing of the error response with its ID
            Logger.log "Queued error response as `#{id}`", origin: @origin, verbosity: 4
          # If the original message was already an error response, do not send another
          elsif @is_error_response == 1
            Logger.log "Original message was an error response; not sending another error response", origin: @origin, verbosity: 4
          end
        end
      end
    end
  end
end
```

lib/mlsmtp/email/error_email.rb

```ruby
# This module defines an SMTP server related namespace with email handling classes
module SMTPServer
  module Email
    # This class constructs and manages error notification emails
    class ErrorEmail
      # Initializes an ErrorEmail object with original sender, recipient, and email name
      # Inputs:
      # - original_from: the original sender email address
      # - original_to: the original recipient email address
      # - email_name: the key name for selecting email template path
      def initialize(original_from:, original_to:, email_name:)
        @og_from = original_from # Store original sender address
        @og_to = original_to     # Store original recipient address
        @contact_email = Config.active["contact_email"] # Fetch contact email from configuration
        @server_name = Config.active["banner"]["banner_server_name"] # Fetch server name from configuration
        @email_name = email_name # Store email template key name
      end

      # Generates the raw original email text with placeholders replaced
      # Returns the email text string or false if file not found
      def raw_original_text
        # Retrieve the file path for the email template based on email_name
        path = Config.active["error_emails"][@email_name]
        # Return false if path is invalid or file does not exist
        return false unless path && File.exist?(path)

        # Read the email template file content
        File.read(path)
          # Replace placeholder for server name with actual server name
          .gsub("<<!SERVERNAME!>>", @server_name)
          # Replace placeholder for contact email with actual contact email
          .gsub("<<!CONTACTADDR!>>", @contact_email)
          # Replace placeholder for sender address with original sender address
          .gsub("<<!SENDERADDR!>>", @og_from)
          # Replace placeholder for recipient address with original recipient address
          .gsub("<<!RECIPIENT!>>", @og_to)
      end

      # Prepares the email string for sending
      # Returns the email as a string or false if raw email text is unavailable
      def prepared_email
        email_text = raw_original_text # Get the processed email template
        # Return false if email template could not be loaded
        return false unless email_text

        # Parse the email text into a Mail object and convert back to string
        Mail.read_from_string(email_text).to_s
      end

      # Read-only accessors for original sender, recipient, contact email, and server name
      attr_reader :og_from, :og_to, :contact_email, :server_name
    end
  end
end
```

lib/mlsmtp/email/email_preparer.rb

```ruby
# Define the SMTPServer module to encapsulate related classes and modules
module SMTPServer
  # Define the Email module within SMTPServer to organize email-related classes
  module Email
    # Define the EmailPreparer class responsible for preparing email messages
    class EmailPreparer
      # Initialize with smtp_context, parse raw email data
      # Inputs: smtp_context object containing email data
      def initialize(smtp_context)
        @context = smtp_context # Store the context object
        @raw_email = @context.data # Extract raw email string from context
        @parsed_email = Mail.read_from_string(@raw_email) # Parse raw email into Mail object
      end

      # Set a specific header in the email
      # Inputs: header name as string or symbol, value as string
      def set_header(h, v)
        @parsed_email.header[h] = v # Assign the header value
      end

      # Add headers to the email based on configuration
      def add_all_headers
        # Iterate over each header and its adapter specified in configuration
        Config.active["header_adapters"].each do |header, adapter|
          # Instantiate the adapter class dynamically using its name
          value = Object.const_get(adapter).new(@context)
          next unless value # Skip if adapter instantiation returns nil
          set_header(header, value.to_s) # Set the header with adapter's string representation
        end
      end

      # Convert the email object back to string format
      def to_s
        @parsed_email.to_s # Return string representation of the email
      end

      # Create attribute readers for context, raw_email, and parsed_email
      attr_reader :context, :raw_email, :parsed_email
    end
  end
end
```

lib/mlsmtp/errors/invalid_command_error.rb

```ruby
# Define a module SMTPServer to encapsulate SMTP server related classes and errors
module SMTPServer
  # Define a nested module Errors to group error classes related to SMTPServer
  module Errors
    # Define a custom error class for invalid SMTP commands, inheriting from StandardError
    class InvalidCommandError < StandardError
      # Initialize the error with the invalid command that caused the error
      def initialize(command)
        @command = command  # Store the invalid command in an instance variable
      end

      # Override the string representation of the error to include the invalid command
      def to_s
        "Command `#{command}` is not a valid command"  # Return error message with command
      end

      # Create an accessor for the command attribute to allow reading and writing
      attr_accessor :command
    end
  end
end
```

lib/mlsmtp/errors/incomplete_command_error.rb

```ruby
# Define the SMTPServer module to encapsulate server-related classes and modules
module SMTPServer
  # Define the Errors module to group custom error classes related to SMTP server operations
  module Errors
    # Define a custom error class for incomplete commands inheriting from StandardError
    class IncompleteCommandError < StandardError
      # Initialize the error with the original command and the expected template command
      # error is expected to be an array containing the original command and the template command
      def initialize(error)
        # Assign the first element of error to original_command
        @original_command, @template_command = error
      end

      # Override the to_s method to provide a descriptive error message
      def to_s
        # Return a string indicating the command is incomplete and does not match the template
        "Command `#{@original_command}` is incomplete (does not match template `#{template_command}`)"
      end

      # Create getter and setter methods for original_command and template_command
      attr_accessor :original_command, :template_command
    end
  end
end
```

lib/mlsmtp/errors/bad_code_error.rb

```
# Define the SMTPServer module to encapsulate related classes and modules
module SMTPServer
  # Define the Errors module to group error-related classes
  module Errors
    # Define the BadCodeError class inheriting from StandardError for custom error handling
    class BadCodeError < StandardError
      # Initialize with a values array containing position and example
      def initialize(values)
        @position, @example = values # Store position and example from input array
      end

      # Override to_s method to provide custom error message based on position
      def to_s
        if @position == :fullcode
          # When the entire code is bad
          "Bad code `#{@example}`"
        elsif @position == :missingelements
          # When some elements are missing in the code
          "Missing element(s) #{@example}"
        else
          # When a specific element in the code is invalid
          "Bad element `#{@example}` in #{@position} position in status code"
        end
      end

      # Create getter methods for position and example
      attr_reader :position, :example
    end
  end
end
```

lib/mlsmtp/errors/nonexistent_mailbox_error.rb

```
# Define a module SMTPServer to encapsulate SMTP related classes and modules
module SMTPServer
  # Define a nested module Errors to group all custom error classes
  module Errors
    # Define a custom error class for nonexistent mailbox errors inheriting from StandardError
    class NonexistentMailboxError < StandardError
      # Initialize method takes a path argument representing the mailbox path
      def initialize(path)
        @path = path  # Store the mailbox path in an instance variable
      end

      # Override the to_s method to provide a custom error message
      def to_s
        "Mailbox `#{@path}` does not exist."  # Return message indicating the mailbox does not exist
      end

      # Create a reader method for the path attribute
      attr_reader :path
    end
  end
end
```

lib/mlsmtp/errors/server_rejection_error.rb

```
# Define a module SMTPServer to encapsulate SMTP server related classes and errors
module SMTPServer
  # Define a nested module Errors to group all error classes related to SMTPServer
  module Errors
    # Define a custom error class for server rejection errors inheriting from StandardError
    class ServerRejectionError < StandardError
      # Initialize method for the error class, no specific setup needed
      def initialize
      end

      # Override the to_s method to provide a custom error message
      def to_s
        "All available servers rejected the message." # Return a descriptive error message
      end
    end
  end
end
```

lib/mlsmtp/errors/missing_config_setting_error.rb

```
# Define a module SMTPServer to encapsulate related classes and modules
module SMTPServer
  # Define a nested module Errors to group custom error classes
  module Errors
    # Define a custom error class for missing configuration settings
    class MissingConfigSettingError < StandardError
      # Initialize the error with optional list of missing settings
      def initialize(missing_settings = nil)
        @missing_settings = missing_settings # Store missing settings for later reference
      end

      # Override the to_s method to provide a descriptive error message
      def to_s
        if @missing_settings
          # Return message including specific missing settings
          "Required configuration settings are missing: #{@missing_settings}"
        else
          # Return generic message if no specific settings are provided
          "Required configuration settings are missing"
        end
      end

      # Provide a reader for the missing_settings attribute
      attr_reader :missing_settings
    end
  end
end
```

lib/mlsmtp/errors/bad_auth_rule_error.rb

```
# Define a module SMTPServer to encapsulate SMTP related classes and errors
module SMTPServer
  # Define a nested module Errors to group all error classes related to SMTPServer
  module Errors
    # Define a custom error class for handling bad authentication rules
    class BadAuthRuleError < StandardError
      # Initialize the error with the provided authentication rule
      # auth_rule - the authentication rule that caused the error
      def initialize(auth_rule)
        @auth_rule = auth_rule
      end

      # Override the to_s method to provide a descriptive error message
      def to_s
        "Bad auth rule: #{@auth_rule}. Acceptable options are \"allow\", \"deny\""
      end

      # Provide a reader for the auth_rule attribute
      attr_reader :auth_rule
    end
  end
end
```

lib/mlsmtp/database/database.rb

```
module SMTPServer  # Defines the SMTPServer module to encapsulate SMTP server related classes and modules
  module Database    # Defines the Database module inside SMTPServer to handle database connection logic
    @@active = nil     # Class variable to hold the current active database connection, initialized as nil

    # Defines a class method to establish a database connection
    def self.connect
      # Retrieves database configuration hash from the active configuration
      database_config = Config.active["database"]["config"]
      # Retrieves the database adapter name (e.g., 'Postgres', 'MySQL') from configuration
      adapter = Config.active["database"]["adapter"]
      # Instantiates a new database connection object using the adapter class with the configuration
      @@active = Object.const_get(adapter).new(database_config)
    end

    # Defines a class method to access the current active database connection
    def self.active
      @@active  # Returns the current active database connection object or nil if not connected
    end
  end
end
```

lib/mlsmtp/database/sqlite.rb

```ruby
module SMTPServer
  module Database
    # This class provides an adapter for SQLite3 database operations within the SMTPServer module
    class SQLite3Adapter
      # Initializes the adapter with configuration settings
      # settings - a hash containing database configuration options
      def initialize(settings)
        @path = settings["path"] # Path to the SQLite3 database file
        @setup_file = settings["db_setup"] # Path to the SQL setup file
        @busy_timeout = settings["busy_timeout"] # Timeout duration for database busy states
        initialize_sqlite # Establish the database connection
        setup_database if settings["autocreate_db"] # Create database schema if auto-creation is enabled
      end

      # Executes a SQL statement with optional parameters
      # *sql - a list of SQL statements or query components
      def exec_sql(*sql)
        @database.execute *sql # Run the SQL command(s) on the database
      end

      # Sets up the database schema by executing SQL commands from setup file
      def setup_database
        @database.execute_batch File.read(@setup_file) # Read and execute batch SQL commands from setup file
      end

      private

      # Sets the busy timeout for database operations to prevent locking issues
      # timeout - duration in milliseconds to wait when the database is busy
      def set_busy_timeout(timeout)
        exec_sql("PRAGMA busy_timeout = #{timeout};") # Execute PRAGMA command to set busy timeout
      end

      # Initializes the SQLite3 database connection and configures busy timeout
      def initialize_sqlite
        @database = SQLite3::Database.open @path # Open connection to the SQLite3 database file
        set_busy_timeout(@busy_timeout) # Configure busy timeout setting
      end
    end
  end
end
```

lib/mlsmtp.rb

```ruby
# Change working directory to the parent directory of the current script's directory
Dir.chdir(File.expand_path('..', __dir__))

# Require the OpenSSL library for cryptographic functions
require "openssl"
# Require a custom patch for OpenSSL object identifier safe registration
require_relative "mlsmtp/patches/openssl_oid_safe_register.rb"

# Require various libraries necessary for email handling and network operations
require "spf"            # Sender Policy Framework for SPF validation
require "dkim"           # DomainKeys Identified Mail for email authentication
require "rpam"           # RPAM library for PAM authentication
require "json"           # JSON parsing and generation
require "mail"           # Mail gem for email message handling
require "base64"         # Base64 encoding and decoding
require "ipaddr"         # IP address manipulation
require "rbtext"         # Text formatting library
require "rbtext/string_methods" # String methods extension for rbtext
require "socket"         # Low-level networking interface
require "resolv"         # DNS resolution
```

```
require "sqlite3"        # SQLite database interface
require "maildir"        # Maildir email storage format
require "argparse"       # Command-line argument parsing

# Define the SMTPServer module to encapsulate server configuration and methods
module SMTPServer
  # List of files that do not require explicit require statements
  @@norequires = [
    "patches/openssl_oid_safe_register.rb",
  ]

  # List of files to preload before main server startup
  @@preload_files = [
    "errors/bad_code_error.rb",
    "errors/missing_config_setting_error.rb",
    "errors/incomplete_command_error.rb",
    "errors/invalid_command_error.rb",
    "errors/nonexistent_mailbox_error.rb",
    "errors/server_rejection_error.rb",
    "errors/bad_auth_rule_error.rb",
    "config.rb",
  ]

  # List of core server and handler files to load during server operation
  @@normal_files = [
    "smtp/commands.rb",
    "smtp/responses.rb",
    "smtp/banner.rb",
    "smtp/smtp_client_context.rb",
    "smtp/smtp_server_context.rb",
    "servers/mailserver.rb",
    "handlers/generic_client_handler.rb",
    "handlers/smtp_server_handler.rb",
    "handlers/smtp_command_handlers/ehlo.rb",
    "handlers/smtp_command_handlers/starttls.rb",
    "handlers/smtp_command_handlers/helo.rb",
    "handlers/smtp_command_handlers/mailfrom.rb",
    "handlers/smtp_command_handlers/rcptto.rb",
    "handlers/smtp_command_handlers/data.rb",
    "handlers/smtp_command_handlers/quit.rb",
    "handlers/smtp_command_handlers/rset.rb",
    "handlers/smtp_command_handlers/vrfy.rb",
    "handlers/smtp_command_handlers/expn.rb",
    "handlers/smtp_command_handlers/noop.rb",
    "handlers/smtp_command_handlers/auth.rb",
    "handlers/smtp_server_command_handlers/ehlo.rb",
    "handlers/smtp_server_command_handlers/helo.rb",
    "handlers/smtp_server_command_handlers/starttls.rb",
    "handlers/smtp_server_command_handlers/quit.rb",
    "handlers/smtp_server_command_handlers/mailfrom.rb",
    "handlers/smtp_server_command_handlers/rcptto.rb",
    "handlers/smtp_server_command_handlers/data.rb",
    "transport/rules.rb",
    "transport/authorization.rb",
    "transport/authorization_handler.rb",
    "transport/destination.rb",
    "transport/local_delivery_agent.rb",
    "transport/remote_delivery_agent.rb",
    "storage/maildir.rb",
    "database/sqlite.rb",
    "queue/queued_message.rb",
    "queue/queue_worker.rb",
    "queue/queue_handler.rb",
    "email/email_preparer.rb",
    "email/headers/received.rb",
    "email/error_email.rb",
```

```
    "email/error_email.rb",
    "email/error_email_generator.rb",
    "logger/logger.rb",
    "ssl/certificates.rb",
    "authentication/pam.rb",
    "authentication/debug.rb",
    "authentication/none.rb",
    "authentication/login_handler.rb",
    "authentication/plain_handler.rb",
    "mail_lists/mail_list.rb",
    "message_authorization/spf/authorize_spf.rb",
    "message_authorization/spf/header.rb",
    "message_authorization/dkim/dkim_signature_header.rb",
  ]
  # List of adapter files for storage, database, and authentication
  @@adapters = [
    "storage/storage.rb",
    "database/database.rb",
    "authentication/auth_adapter.rb",
  ]
  # Additional configuration and email template files
  @@additional_files = [
    "conf_templates/default.json",
    "conf_templates/transport_rules.json",
    "conf_templates/transport_authorization.json",
    "conf_templates/dkim_maps.json",
    "conf/initialize_db.sql",
    "emails/bad_mailbox.eml",
    "emails/delivery_failed.eml"
  ]
  # List of executable scripts related to the SMTP server
  @@exe = [
    "mlsmtpd",
    "mlsmtplist",
    "mlsmtpqueue",
    "mlsmtpnewinstance"
  ]

  # Method to return the current version of the SMTPServer module
  def self.version
    "0.0.1"
  end

  # Method to retrieve the list of additional files
  def self.additional_files
    @@additional_files
  end

  # Method to retrieve the list of executable scripts
  def self.executables
    @@exe
  end

  # Method to retrieve the list of normal server files
  def self.normal_files
    @@normal_files
  end

  # Method to retrieve the list of adapter files
  def self.adapters
    @@adapters
  end

  # Method to retrieve the list of preload files
  def self.preload_files
    @@preload_files
```

```
  end

  # Method to retrieve the list of files that do not require explicit require
  def self.norequires
    @@norequires
  end

  # Method to generate full file paths for loading based on relative flag
  # If relative is true, returns paths relative to the lib directory
  # Otherwise, returns absolute paths including the main mlsmtp.rb file
  def self.file_paths(relative:false)
    x = (@@normal_files + @@adapters + @@norequires).map do |f|
      "#{"lib/" unless relative}mlsmtp/#{f}"
    end

    if relative
      return x
    else
      return x + ['lib/mlsmtp.rb']
    end
  end

  # Method to load preload files using require_relative
  def self.load_preload
    preload_files.each do |f|
      require_relative "mlsmtp/#{f}"
    end
  end

  # Method to load remaining core files and adapters after preload
  def self.load_remaining
    normal_files.each do |f|
      require_relative "mlsmtp/#{f}"
    end

    # Require additional external dependencies specified in active configuration
    Config.active["additional_requires"].each do |r|
      require r
    end

    # Load adapter files
    adapters.each do |f|
      require_relative "mlsmtp/#{f}"
    end

    # Set thread behavior based on configuration for exception handling
    Thread.abort_on_exception = SMTPServer::Config.active["threads"]["abort_on_exception"]
    Thread.report_on_exception = SMTPServer::Config.active["threads"]["report_on_exception"]
  end
end
```

bin/mlsmtpqueue

```
#!/usr/bin/env ruby
# Require necessary libraries for SMTP server management and argument parsing
require "mlsmtp"
require "argparse"

# Load preloaded SMTP server configurations or data
SMTPServer.load_preload

# Define a method to print details of a queued message given an array of message attributes
def print_queued_message(message_arr)
  # Unpack message attributes into individual variables
  mid, uid, is_smtp_response, created_at, mail_from, rcpt_to, file_path, retries, try_at = message_arr
```

```
  mid, uid, is_error_response, created_at, mail_from, rcpt_to, file_path, retries, try_at = message_arr

  # Output formatted message details to standard output
  STDOUT.puts <<QM
---
Message ID: #{mid}
Unique ID: #{uid}
Error Response: #{is_error_response}
Queued At: #{created_at}
Attempt At: #{try_at}
Retries Remaining: #{retries}
Mail From: #{mail_from}
Rcpt To: #{rcpt_to}
File Path: #{file_path}
QM
end

# Define options for command line argument parsing
options = {
  conf: { has_argument: true }, # Configuration file argument
  help: {},                     # Help flag
  eqmod: { has_argument: true },# Equal modulo argument for filtering
  unqueue_uid: { has_argument: true }, # Unqueue message by UID
  mid: { has_argument: true },  # Message ID argument
  uid: { has_argument: true },  # UID argument
  count: {},                    # Count messages in queue
  clear: {}                     # Clear all queued messages
}

# Define switches if any (empty in this case)
switches = {

}

# Parse command line arguments using ArgsParser
args = ArgsParser::Args.new(options: options, switches: switches)

# If help option is specified, display help text and exit
if args.options[:help]
  help_text = <<~TEXT
    MLSMTP Email List Manager
    Usage: mlsmtpqueue <list> [options]

    Options:
      --conf: specify MLSMTP configuration file
      --help: print this help menu
      --eqmod <eq>:<mod>: list queued messages where mid mod equals eq
      --unqueue_uid <uid>: unqueue message with UID <uid>
      --mid <mid>: display message with MID <mid>
      --uid <uid>: display message with UID <uid>
      --count: only display number of messages in queue
      --clear: clear all queued messages
  TEXT

  STDOUT.puts help_text
  exit
end

# Parse eqmod option if provided, split into eq and mod integers
eq, mod = args.options[:eqmod] ? args.options[:eqmod].split(?:).map(&:to_i) : [ 0, 1 ]

# Determine configuration file path, defaulting to default.json
config_file = args.options[:conf] || "conf/default.json"

# Validate that configuration file path is a string
unless config_file.is_a?(String)
```

```
  STDERR.puts "Specify a configuration file in the --conf parameter."
  exit 1
end

# Check if configuration file exists
unless File.exist?(config_file)
  STDERR.puts "#{config_file} does not exist."
  exit 1
end

# Load configuration from file and set it as active, handle potential errors
begin
  active_config = SMTPServer::Config.from_file(config_file)
  active_config.set_active
rescue JSON::ParserError => e
  # Log JSON parsing errors
  SMTPServer::Logger.log "Error parsing JSON configuration file.", type: :error, origin: "MLSMTP List Manager"
  exit 1
rescue SMTPServer::Errors::MissingConfigSettingError => e
  # Log missing configuration settings
  SMTPServer::Logger.log "Required setting(s): #{e.missing_settings} are missing.", type: :error, origin: "MLSMTP List Manager"
  exit 1
end

# Load remaining SMTP server data after configuration
SMTPServer.load_remaining

# Establish connection to the SMTP server database
SMTPServer::Database.connect

# If unqueue UID option is specified, unqueue that message and exit
if args.options[:unqueue_uid]
  SMTPServer::Queue::QueuedMessage.unqueue_uid(args.options[:unqueue_uid].to_i)
  exit
end

# If clear option is specified, unqueue all messages that match the filter
if args.options[:clear]
  SMTPServer::Queue::QueuedMessage.find_by_mod(mod: 1, eq: 0).each do |m|
    SMTPServer::Queue::QueuedMessage.unqueue_uid(m[1])
  end
end

# If message ID is specified, find and display that message, then exit
if args.options[:mid]
  print_queued_message(
    SMTPServer::Queue::QueuedMessage.find_my_mid(args.options[:mid].to_i).to_a
  )
  exit
end

# If UID is specified, find and display that message, then exit
if args.options[:uid]
  print_queued_message(
    SMTPServer::Queue::QueuedMessage.find_my_uid(args.options[:uid].to_i).to_a
  )
  exit
end

# Retrieve queued messages filtered by mod and eq values
queued_messages = SMTPServer::Queue::QueuedMessage.find_by_mod(mod: mod, eq: eq)

# If count option is specified, output total number of queued messages and exit
if args.options[:count]
  STDOUT.puts "#{queued_messages.count} queued message(s)"
```

```
  exit
end

# For each queued message retrieved, print its details
queued_messages.each { |qm| print_queued_message(qm) }
```

bin/mlsmtplist

```
#!/usr/bin/env ruby
# This script is a command-line email list manager using the MLSMTP library and Ruby language.

require "mlsmtp" # Load MLSMTP library for email list management functionalities.
require "argparse" # Load Argparse library for command-line argument parsing.

SMTPServer.load_preload # Preload SMTPServer configurations and dependencies.

# Define command-line options with their argument requirements.
options = {
  conf: { has_argument: true }, # Configuration file path option.
  create: {}, # Create a new email list.
  delete: {}, # Delete an existing email list.
  listall: {}, # List all email lists.
  listmembers: {}, # List members of a specific email list.
  addmember: { has_argument: true }, # Add a member to a list.
  removemember: { has_argument: true }, # Remove a member from a list.
  help: {} # Show help message.
}

# Define command-line switches (none in this case).
switches = {

}

# Parse command-line arguments based on options and switches.
args = ArgsParser::Args.new(options: options, switches: switches)

# If help option is specified, display usage information and exit.
if args.options[:help]
  help_text = <<~TEXT
    MLSMTP Email List Manager
    Usage: mlsmtplist <list> [options]

    Options:
      --conf: specify MLSMTP configuration file
      --help: print this help menu
      --listall: list all email lists
      --create: create list <list>
      --delete: delete list <list>
      --listmembers: list members in <list>
      --addmember <member>: add <member> to <list>
      --removemember <member>: remove <member> from <list>
  TEXT

  STDOUT.puts help_text # Output help message.
  exit # Exit program after displaying help.
end

# Retrieve the first positional argument as the list name.
list = args.data[0].to_s

# Determine the configuration file, defaulting if not specified.
config_file = args.options[:conf] || "conf/default.json"

# Check if list name is empty and listall option isn't set.
if list.gsub(/[\s]/, "") == "" && !args.options[:listall]
```

```
  STDERR.puts "No list specified" # Output error if no list provided.
  exit 2 # Exit with error code 2.
end

# Verify that the configuration file parameter is a string.
unless config_file.is_a?(String)
  STDERR.puts "Specify a configuration file in the --conf parameter." # Error message.
  exit 1 # Exit with error code 1.
end

# Check if the configuration file exists.
unless File.exist?(config_file)
  STDERR.puts "#{config_file} does not exist." # Error if file missing.
  exit 1 # Exit with error code 1.
end

# Load and set the active SMTP configuration from the file.
begin
  active_config = SMTPServer::Config.from_file(config_file) # Parse config file.
  active_config.set_active # Activate the configuration.
rescue JSON::ParserError => e
  SMTPServer::Logger.log "Error parsing JSON configuration file.", type: :error, origin: "MLSMTP List Manager"
  exit 1 # Exit if JSON parsing fails.
rescue SMTPServer::Errors::MissingConfigSettingError => e
  SMTPServer::Logger.log "Required setting(s): #{e.missing_settings} are missing.", type: :error, origin: "MLSMTP List Manager"
  exit 1 # Exit if required settings are missing.
end

# Load any remaining SMTPServer dependencies or settings.
SMTPServer.load_remaining

# Establish connection to the SMTPServer database.
SMTPServer::Database.connect

# If listall option is specified, list all email lists and exit.
if args.options[:listall]
  STDOUT.puts SMTPServer::MailLists::MailList.all.map{|a| "#{a[0]}: #{a[1]}"} # Output list name and description.
  exit 0 # Exit successfully.
end

# If create option is specified, create a new email list.
if args.options[:create]
  SMTPServer::MailLists::MailList.create(list)
end

# If delete option is specified, delete the specified email list.
if args.options[:delete]
  SMTPServer::MailLists::MailList.find_by_name(list)&.delete
end

# If listmembers option is specified, list members of the specified list.
if args.options[:listmembers]
  STDOUT.puts SMTPServer::MailLists::MailList.find_by_name(list)&.expand
end

# If addmember option is specified, add a member to the list.
if args.options[:addmember]
  SMTPServer::MailLists::MailList.find_by_name(list)&.add_membership(args.options[:addmember])
end

# If removemember option is specified, remove a member from the list.
if args.options[:removemember]
  SMTPServer::MailLists::MailList.find_by_name(list)&.remove_membership(args.options[:removemember])
end
```

bin/mlsmtpnewinstance

```
#!/usr/bin/env ruby
require "argparse" # Require the argument parsing library

# Initialize options hash with help key for storing help option state
options = {
  help: {} # Placeholder for help option
}

# Initialize switches hash, currently empty, can be used for command-line switches
switches = {

}

# Create an args object using ArgsParser with specified options and switches
args = ArgsParser::Args.new(options: options, switches: switches)

# Check if help option is requested
if args.options[:help]
  # Define help text explaining script usage
  help_text = <<~TEXT
    Create new MLSMTP instance
    Usage: mlsmtpnewinstance <directory>
  TEXT

  # Output help text to standard output
  STDOUT.puts help_text
  # Exit the script after displaying help
  exit
end

# Retrieve the first positional argument which should be the directory
directory = args.data[0]

# If directory argument is missing, output error and exit
unless directory
  STDERR.puts "You must specify a directory."
  exit 1
end

# Create the directory if it does not exist
Dir.mkdir(directory) unless File.directory?(directory)

# Check if the specified directory path is actually a file, not a directory
if File.file?(directory)
  STDERR.puts "#{directory} is a file." # Output error if path is a file
  exit 1
end

# Loop through all files in the configuration templates directory
Dir.glob(File.expand_path("..", __dir__) + "/conf_templates/*").each do |file|
  # Construct the destination path within the target directory
  path = "#{directory}/#{File.basename(file)}"

  # Read content from the template file and write it to the new location
  File.write(
    path,
    File.read(file)
  )

  # Output the path of the created file
  STDOUT.puts path
end
```

bin/mlsmtpd

```
#!/usr/bin/env ruby
# Require the mlmtp library for SMTP server functionalities
require "mlsmtp"
# Require the argparse library for command line argument parsing
require "argparse"

# Load any preloaded SMTP server settings or modules
SMTPServer.load_preload

# Define a method to handle graceful shutdown of the server
def graceful_shutdown()
  # Log shutdown message with origin as Daemon
  SMTPServer::Logger.log "Shutting down...", origin: "Daemon"
  # Kill all running mail servers
  SMTPServer::Servers::MailServer.all_servers.dup.each(&:kill)
  # Kill all queue workers
  SMTPServer::Queue::QueueWorker.all_workers.dup.each(&:kill)
  # Exit the program
  exit
end

# Set up a trap for interrupt signals (e.g., Ctrl+C) to invoke graceful shutdown
trap("INT") { graceful_shutdown }

# Define command line options for the script
options = {
  conf: { has_argument: true }, # Option for configuration file path
  help: {} # Help option
}

# Define switches if any (empty in this case)
switches = {

}

# Parse command line arguments with options and switches
args = ArgsParser::Args.new(options: options, switches: switches)

# If help option is specified, display help text and exit
if args.options[:help]
  help_text = <<~TEXT
    MLSMTP Daemon
    Usage: mlsmtpd [--conf <file>]

    Options:
      --conf: specify MLSMTP configuration file
      --help: print this help menu
  TEXT

  STDOUT.puts help_text
  exit
end

# Set configuration file path from arguments or default to conf/default.json
config_file = args.options[:conf] || "conf/default.json"

# Check if configuration file is a string
unless config_file.is_a?(String)
  STDERR.puts "Specify a configuration file in the --conf parameter."
  exit 1
end

# Check if configuration file exists
```

```
unless File.exist?(config_file)
  STDERR.puts "#{config_file} does not exist."
  exit 1
end

# Load and parse configuration file, handle errors
begin
  active_config = SMTPServer::Config.from_file(config_file)
  # Set the loaded configuration as active
  active_config.set_active
rescue JSON::ParserError => e
  # Log JSON parsing errors
  SMTPServer::Logger.log "Error parsing JSON configuration file.", type: :error, origin: "Daemon"
  exit 1
rescue SMTPServer::Errors::MissingConfigSettingError => e
  # Log missing configuration settings
  SMTPServer::Logger.log "Required setting(s): #{e.missing_settings} are missing.", type: :error, origin: "Daemon"
  exit 1
end

# Load remaining SMTP server components after configuration
SMTPServer.load_remaining

# Retrieve server configurations from the active configuration
servers = active_config["servers"]
# Iterate through each server configuration
servers.each do |server|
  # Extract host, port, encryption, certificate, and autostart settings
  host = server["host"]
  port = server["port"]
  encryption = server["encryption"].to_sym
  certificate = SMTPServer::SSL::Certificate[server["certificate"]]
  autostart = server["autostart"]

  # Create a new SMTP mail server instance with the specified settings
  s = SMTPServer::Servers::MailServer.new(host: host, port: port, encryption: encryption, certificate: certificate)

  # Log creation of the server with its object ID and connection details
  SMTPServer::Logger.log "Created server #{s.object_id} on #{host}:#{port} with encryption #{encryption}", origin: "Daemon"
  # If autostart is enabled, start the server
  if autostart
    s.start
    # Log that the server has started with its process ID
    SMTPServer::Logger.log "Started server #{s.object_id} PID: #{s.pid}", origin: "Daemon"
  end
end

# Retrieve the number of queue worker threads from configuration
queue_workers_count = active_config["queue"]["workers"]["amount"]
# Loop to create and start each queue worker
queue_workers_count.times do |i|
  # Instantiate a new queue worker with module and index
  worker = SMTPServer::Queue::QueueWorker.new(
    mod: queue_workers_count,
    eq: i
  )

  # Log creation of the worker with its index and queue position
  SMTPServer::Logger.log "Created worker #{i} (#{worker.eq}:#{worker.mod})", origin: "Daemon"

  # Start the worker thread
  worker.start

  # Log that the worker has started with its process ID
  SMTPServer::Logger.log "Started worker #{i} PID: #{worker.pid}", origin: "Daemon"
end
```

```
# Enter an infinite loop to keep the daemon running
loop{}
```

mlsmtp.gemspec

```
 # Require the relative path to the library file SMTPServer.rb which contains server logic
require_relative "lib/mlsmtp.rb"

# Define a new gem specification object for the mlsmtp gem
Gem::Specification.new do |mlsmtp|
  # Set the name of the gem
  mlsmtp.name        = 'mlsmtp'
  # Set the version of the gem based on SMTPServer's version
  mlsmtp.version     = SMTPServer.version
  # Provide a brief summary of the gem
  mlsmtp.summary     = "Ruby SMTP server"
  # Provide a detailed description of the gem
  mlsmtp.description = "Highly configurable and flexible SMTP server written in pure ruby"
  # List the authors of the gem
  mlsmtp.authors     = ["Matthias Lee"]
  # Provide contact email
  mlsmtp.email       = 'matthias@matthiasclee.com'
  # Specify the files to include in the gem by combining preload files, file paths, executables, and additional files
  mlsmtp.files       = SMTPServer.preload_files.map{|x|"lib/mlsmtp/#{x}"} + SMTPServer.file_paths + SMTPServer.executables.map{|i|"b
  # Specify the executable files for the gem
  mlsmtp.executables = SMTPServer.executables
  # Set the path where Ruby should look for required files
  mlsmtp.require_paths = ['lib']
  # Specify the minimum required Ruby version for this gem
  mlsmtp.required_ruby_version = ">= 3.0"

  # Add runtime dependencies with version constraints to ensure compatibility
  mlsmtp.add_runtime_dependency "spf", "~> 0.1.1"
  mlsmtp.add_runtime_dependency "dkim", "~> 1.1.0"
  mlsmtp.add_runtime_dependency "json", "~> 2.6.3"
  mlsmtp.add_runtime_dependency "mail", "~> 2.8.1"
  mlsmtp.add_runtime_dependency "ipaddr", "~> 1.2.7"
  mlsmtp.add_runtime_dependency "rbtext", "~> 0.3.5"
  mlsmtp.add_runtime_dependency "resolv", "~> 0.3.0"
  mlsmtp.add_runtime_dependency "sqlite3", "~> 2.6.0"
  mlsmtp.add_runtime_dependency "maildir", "~> 2.2.3"
  mlsmtp.add_runtime_dependency "openssl", "~> 3.1.0"
  mlsmtp.add_runtime_dependency "argparse", "~> 0.0.5"
  mlsmtp.add_runtime_dependency "rpam-ruby19", "~> 1.2.1"

  # Set the homepage URL for the project
  mlsmtp.homepage = 'https://github.com/Matthiasclee/mlsmtp'
  # Specify the license type for the gem
  mlsmtp.license = 'CC-BY-NC-SA-4.0'
end
```

emails/bad_mailbox.eml

```
 Content-Type: text/plain;
From: <<!SERVERNAME!>> Delivery Agent <<<!CONTACTADDR!>>>
Subject: Email Delivery Failed
Content-Language: en-US
To: <<!SENDERADDR!>>

<<!SERVERNAME!>> was unable to deliver your message to <<!RECIPIENT!>>, as that email address does not exist on this server. Contact
```

emails/delivery_failed.eml

```
Content-Type: text/plain;
From: <<!SERVERNAME!>> Delivery Agent <<<!CONTACTADDR!>>>
Subject: Email Delivery Failed
Content-Language: en-US
To: <<!SENDERADDR!>>

<<!SERVERNAME!>> was unable to deliver your message to <<!RECIPIENT!>>, as all of the servers either were not available or rejected
```

conf_templates/transport_authorization.json

```
[
  {
    "rule": "allow",
    "auth_exempt": true,
    "match_by": {
      "from_ip": [ "127.0.0.1/32", "::1/128" ]
    },
    "determine_by": {
    }
  },
  {
    "rule": "allow",
    "auth_exempt": true,
    "match_by": {
      "from_email": "^.*@localhost$"
    },
    "determine_by": {
      "auth": "%u"
    }
  },
  {
    "rule": "allow",
    "match_by": {
      "to_email": "^.*@localhost$"
    },
    "determine_by": {
    }
  }
]
```

conf_templates/dkim_maps.json

```
{
  "localhost": {
    "keypair": "default",
    "domain": "localhost",
    "selector": [ "default" ]
  }
}
```

conf_templates/transport_rules.json

```
{
  "^.*@localhost$": [ "%u" ],
  "^.*@use_proxy.local$": [ "%a", [ "proxy1.local" ] ]
}
```

conf_templates/default.json

```
{
  "mailname": "localhost",
  "esmtp_enable": true,
  "require_helo": true,
  "contact_email": "postmaster@localhost",
  "dkim_maps": "conf/dkim_maps.json",
  "max_list_recursion": 0,
  "transport": {
    "rules_file": "conf/transport_rules.json",
    "authorization_file": "conf/transport_authorization.json",
    "delivery_timeout": 5,
    "max_retries": 10,
    "retry_interval": 600
  },
  "support_8_bit": true,
  "max_size": 10000000000,
  "disable_commands": [
  ],
  "certificates": {
    "default": {
      "cert_path": "ssl/server.crt",
      "key_path": "ssl/server.key"
    },
    "dkim": {
      "key_path": "ssl/dkim.key"
    }
  },
  "queue": {
    "workers": {
      "amount": 1,
      "restart_delay": 0.2
    },
    "queued_mail_dir": "data/queued_mail",
    "remove_on_unqueue": true,
    "return_queue_ids": true
  },
  "error_emails": {
    "delivery_failed": "emails/delivery_failed.eml",
    "bad_mailbox": "emails/bad_mailbox.eml"
  },
  "logger": {
    "log_to_stdout": true,
    "log_to_files": [],
    "stdout_colors": true,
    "time_format": "%m/%d/%Y %H:%M %Z",
    "max_verbosity_level": -1
  },
  "database": {
    "adapter": "SMTPServer::Database::SQLite3Adapter",
    "config": {
      "path": "data/mlsmtp.db",
      "db_setup": "conf/initialize_db.sql",
      "busy_timeout": 5000,
      "autocreate_db": true
    }
  },
  "header_adapters": {
    "Received": "SMTPServer::Email::Headers::Received",
    "Received-SPF": "SMTPServer::MessageAuthorization::SPFHeader",
    "DKIM-Signature": "SMTPServer::MessageAuthorization::DKIMSignatureHeader"
  },
  "mail_storage": {
    "adapter": "SMTPServer::Storage::MaildirAdapter",
    "config": {
      "path": "data/maildir/%u",
      "create_mailboxes": false
```

```
    "create_mailboxes": false
  }
},
"banner": {
  "include_esmtp_status": true,
  "banner_server_name": "MLSMTP",
  "additional_text": null,
  "message_override": false
},
"socket": {
  "restart_delay": 0.2
},
"threads": {
  "abort_on_exception": false,
  "report_on_exception": false
},
"servers": [
  {
    "host": "0.0.0.0",
    "port": 2525,
    "encryption": "starttls",
    "certificate": "default",
    "autostart": true
  },
  {
    "host": "0.0.0.0",
    "port": 5870,
    "encryption": "starttls",
    "certificate": "default",
    "autostart": true
  },
  {
    "host": "0.0.0.0",
    "port": 4650,
    "encryption": "implicit",
    "certificate": "default",
    "autostart": true
  }
],
"external_transport": {
  "attempt_ports": [
    25,
    2525
  ],
  "socket_timeout": 3
},
"authentication": {
  "adapter": "SMTPServer::Authentication::DebugAdapter",
  "config": {
    "credentials": {
      "user": "pass"
    }
  },
  "valid_auth_methods": {
    "LOGIN": [
      "SMTPServer::Authentication",
      "method_login_handler"
    ],
    "PLAIN": [
      "SMTPServer::Authentication",
      "method_plain_handler"
    ]
  }
},
"postdata_authorization_adapters": {
  "SMTPServer::MessageAuthorization::SPFAuthenticator#authorize_spf": {
```

```
      "versions": [1, 2],
      "ok_results": [
        "pass",
        "none",
        "neutral",
        "softfail"
      ],
      "on_spf_fail": "none"
    }
  },
  "additional_requires": [
  ]
}
```